IMPLEMENTATION_PLAYBOOK

Helios â€” Implementation Playbook

`IMPLEMENTATION_PLAYBOOK.md` Â· The standalone build manual Â· **Version:** v1.0 Â· **Date:** 2026-06-03

> **Read this first.** This manual assumes you have **no further AI assistance** â€” just this repo and your own hands. Everything you need is already here. The production-grade *code and specs already exist in the docs* (full dbt SQL in `DBT_GUIDE.md`, MCP/agent skeletons in the architecture docs, the live `semantic_layer.yaml`, the 50 `eval/scenarios/*.yaml`). This playbook's job is to **sequence the build** and give you, for each milestone, the **objective, the exact files, the commands, the tests, the success criteria, and the mistakes to avoid**. Build the milestones in order. Never skip the golden tests on M3 â€” they fail *silently*.

0.1 What you are building (30-second recap)

Helios is an autonomous growth-diagnosis engine on the GA4 Google Merchandise Store dataset. Raw GA4 events â†' **dbt** marts â†' a **governed semantic layer** â†' **5 MCP servers** (the only paths to SQL and to math) â†' **7 agents** on a deterministic state machine that diagnose *why* the funnel moved (mix-shift vs rate-change), price it in dollars, and ship a Decision Brief â€” graded by a **50-scenario offline benchmark** (target â‰¥85% root-cause accuracy vs â‰¤45% naive baseline).

0.2 Prerequisites & toolchain (do this once)

Need	What
Cloud	A **GCP project** with billing; BigQuery API enabled; read access to `bigquery-public-data` (public, no setup).
Auth	A least-privilege service account (`roles/bigquery.dataViewer` + `roles/bigquery.jobUser`) **or** `gcloud auth application-default login` for dev.
Local	**Python 3.11**, the **gcloud SDK**, **git**, and (optionally) the **GitHub CLI** for CI.
LLM	An **Anthropic API key** (`ANTHROPIC_API_KEY`) for the Claude Agent SDK â€” or any tool-calling LLM behind the same MCP interface (see Â§0.6).

`requirements.txt` (create at repo root in M0):

```
dbt-core≥1.7,<2.0
dbt-bigquery≥1.7,<2.0
google-cloud-bigquery≥3.0
scipy≥1.11
statsmodels≥0.14
prophet≥1.1            # forecasting (or swap pmdarima)
pandas≥2.0
pyyaml≥6.0
mcp≥1.0                # Model Context Protocol SDK
anthropic≥0.39         # or your LLM provider's SDK
pytest≥8.0
```

dbt packages (via `packages.yml`, installed with `dbt deps`): `dbt_utils`, `dbt_expectations`, `dbt_project_evaluator`, `codegen`.

Environment variables (set in your shell / CI secrets):

```
export GOOGLE_APPLICATION_CREDENTIALS=/path/to/sa.json   # or use ADC
export HELIOS_WH_TOKEN=...             # warehouse-mcp bearer (M6)
export ANTHROPIC_API_KEY=...           # agents (M7)
export HELIOS_VECTOR_STORE_URI=...     # memory ANN recall (M8)
```

> Windows note: activate the venv with `.venv\Scripts\activate` (POSIX: `source .venv/bin/activate`). All dbt/gcloud/python commands are otherwise identical.

0.3 How each milestone is written

Every milestone below has the same six parts: **Objective Â· Files to create Â· Commands to run Â· Tests to run Â· Success criteria Â· Common mistakes**. "Files to create" names the path **and the doc + section to copy the code from** â€" you are assembling, not inventing. Files marked *(already exists)* must **not** be regenerated.

0.4 Global build map

Milestone	Builds	Level	Exit gate (one line)
M0	Repo + GCP/IAM + dbt config	L1	`dbt debug` green; bounded query over `events_*` returns rows
M1	Sources, macros, seed	L1	`dbt deps` + `dbt seed`; macros compile
M2	Staging models	L1	`dbt build --select staging`; key tests green
M3 â˜…	Sessionization + funnel keystone	L1	`session_key` unique; **funnel monotonicity** test passes
M4	Marts (core/finance/growth)	L1	`dbt build` green; revenue reconciles to the cent; channels = 10
M5	Semantic layer live	L1/L2	`validate_semantic.py` â†' 0 dangling refs against real marts
M6	semantic-mcp + warehouse-mcp	L1	`build_queryâ†'dry_runâ†'run_queryâ†'reconcile` round-trips; budget caps
M6b	stats / experiment / report MCP	L1/L2	`decompose_change` golden test passes
M7	Minimal autonomous loop	**L1 DONE**	anomaly â†' brief in <5 min; 0 hallucinated columns
M8	Memory store	L2	`save_diagnosis` â†' `recall_prior` round-trips; calendars seeded
M9	Full 7-agent loop	L2	PASS findings carry significance + $ + experiment
M10	Eval harness + CI	**L2 DONE**	â‰¥85% vs â‰¤45%; hallucination = 0; CI gate green
M11	Autonomy & depth	**L3 DONE**	<5 min/run on schedule; accuracy holds all dims
M12	Productionization & frontier	â€"	(deferred) multi-tenant; causal inference

0.5 Target file tree (check each off as you build it)

```
helios/
â”œâ”€ requirements.txt Â· .gitignore Â· dbt_project.yml Â· profiles.yml Â· packages.yml Â· mcp_servers.yaml   [M0]
â”œâ”€ scripts/validate_semantic.py                                                                        [M5]
â”œâ”€ models/
â”‚  â”œâ”€ staging/   src_ga4.yml Â· stg_ga4__events.sql Â· stg_ga4__event_params.sql Â· stg_ga4__schema.yml  [M1â€“M2]
```

```
â”‚  â”œâ”€ intermediate/ int_ga4__sessionized.sql Â· int_ga4__funnel_steps.sql Â· int_ga4__schema.yml        [M3]
â”‚  â”œâ”€ marts/core/ fct_sessions Â· fct_funnel Â· fct_daily_funnel Â· dim_users Â· dim_items Â· dim_channels Â· dim_date (+core__sch
â”‚  â”œâ”€ marts/finance/ fct_orders Â· fct_order_items (+finance__schema.yml)                                [M4]
â”‚  â”œâ”€ marts/growth/ fct_funnel_by_dim Â· fct_cohorts (+growth__schema.yml)                               [M4]
â”‚  â””â”€ semantic/  semantic_layer.yaml (exists) Â· metrics__schema.yml                                    [M5]
â”œâ”€ macros/ get_event_param.sql Â· sessionize.sql Â· channel_group.sql Â· test_revenue_reconciles.sql      [M1]
â”œâ”€ seeds/ channel_group_mapping.csv  Â· snapshots/ snap_dim_items.sql  Â· tests/ assert_*.sql           [M1/M4]
â”œâ”€ sql/ helios_memory_ddl.sql                                                                        [M8]
â”œâ”€ helios/
â”‚  â”œâ”€ mcp/ base.py Â· schemas.py Â· warehouse.py Â· semantic.py Â· stats.py Â· experiment.py Â· report.py    [M6â€“M6b,M8]
â”‚  â”œâ”€ agents/ framework.py Â· orchestrator.py Â· monitor.py Â· decompose.py Â· diagnose.py Â· critic.py Â· prescribe.py Â· narrato
â”‚  â””â”€ runner.py                                                                                       [M7,M9]
â”‚  â””â”€ eval/ injector.py Â· runner.py Â· report.py Â· scorers/*.py Â· scenarios/*.yaml (exist)              [M10]
â”œâ”€ eval/ gates.yaml Â· baselines/main.json                                                           [M10]
â”œâ”€ .github/workflows/ ci.yml                                                                        [M10]
â””â”€ docs/  (the specs you build FROM â€” already exist)
```

0.6 Conventions you must obey (cheat-sheet)

```
session_key = TO_HEX(MD5(CONCAT(user_pseudo_id,'-',CAST(ga_session_id AS STRING))))
```
, one expression everywhere.

- `reached_*` funnel flags are **max-downstream monotonic** â†' step rates â‰¤ 1 by construction (`did_*` is retired).
- Exactly **10** channel groups, defined in **one** macro `channel_group_case()` ; `traffic_source` is user first-touch â€" prefer session-scoped `event_params` source/medium.
- Money: `*_in_usd` only; rates `SUM(num)/SUM(den)` after grouping.
- The LLM **never** writes SQL (only `semantic-mcp`) and **never** computes stats (only `stats-mcp`); `dry_run` before every `run_query` .
- Stats are seeded (`rng_seed = 1729`); per-run byte budget â‰¤ 5 GiB.

If you can't use the Anthropic API: the analytics value (M0â€"M5: governed marts + the semantic layer) is fully usable with no LLM at all. For the agents (M7+), the MCP servers are LLM-agnostic â€" point any tool-calling model at the same `mcp_servers.yaml` interface; only `helios/agents/framework.py` (the model client) changes.

M0 - Foundation & Toolchain

Objective

Stand up the empty Helios repository so that dbt can authenticate to BigQuery, run a bounded smoke query over the GA4 public sample, and report all-green on `dbt debug` . Nothing transforms data yet; M0 only proves the toolchain (Python venv, GCP/IAM/ADC, dbt config, git) is wired correctly and cheaply. This is the foundation every later milestone builds on, so the exit gate is hard: `dbt debug` all-green, ADC authenticates, and a `_TABLE_SUFFIX` -bounded query over `events_*` returns rows under the byte budget.

Files to create

Path	Copy from
`requirements.txt`	exact package list below (StackToolchain)
`.gitignore`	snippet below
`dbt_project.yml`	DBT_GUIDE.md section 1 ("dbt_project.yml") - paste verbatim
`profiles.yml`	DBT_GUIDE.md section 1 ("profiles.yml") - paste verbatim; set `project:` to your GCP project id
`packages.yml`	DBT_GUIDE.md section 1 ("packages.yml") - paste verbatim
`mcp_servers.yaml`	stub only (a single `servers:` key with a TODO comment); finalized in M6

Path	Copy from
`docs/CLAUDE.md`	ALREADY EXISTS - do not regenerate

`requirements.txt` (exact pins for the whole project; M0 only needs the dbt + google rows but pin the rest now so the venv is stable):

```
dbt-core≥1.7,<1.9
dbt-bigquery≥1.7,<1.9
google-cloud-bigquery≥3.17
scipy≥1.11
statsmodels≥0.14
prophet≥1.1
pmdarima≥2.0
pandas≥2.1
pyyaml≥6.0
mcp≥1.0
anthropic≥0.39
pytest≥8.0
```

`.gitignore` (keep secrets and build artifacts out of git):

```
.venv/
target/
dbt_packages/
logs/
*.json            # service-account keys must never be committed
!eval/baselines/*.json
.env
__pycache__/
.user.yml
```

Commands to run

```
# 1. Repo + venv (Windows activation shown; POSIX = source .venv/bin/activate)
git init
python -m venv .venv
.venv\Scripts\activate            # Windows;  POSIX: source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt

# 2. GCP auth via ADC (interactive OAuth for dev — no key files on the laptop)
gcloud auth application-default login
gcloud config set project <PROJECT>          # e.g. helios-analytics

# 3. Least-privilege service account (used by warehouse-mcp / CI later; NOT Owner)
gcloud iam service-accounts create helios-wh \
  --display-name "Helios warehouse read-only"
gcloud projects add-iam-policy-binding <PROJECT> \
  --member "serviceAccount:helios-wh@<PROJECT>.iam.gserviceaccount.com" \
  --role roles/bigquery.dataViewer
gcloud projects add-iam-policy-binding <PROJECT> \
  --member "serviceAccount:helios-wh@<PROJECT>.iam.gserviceaccount.com" \
  --role roles/bigquery.jobUser
# For a server/CI key (only when you need a JSON key; dev uses OAuth ADC above):
#   gcloud iam service-accounts keys create helios-wh.json \
#     --iam-account helios-wh@<PROJECT>.iam.gserviceaccount.com
#   export GOOGLE_APPLICATION_CREDENTIALS=$PWD/helios-wh.json   # never commit this

# 4. dbt: install pinned packages, then prove the connection
dbt deps
dbt debug

# 5. Bounded smoke query over events_* (prune shards with _TABLE_SUFFIX!)
bq query --use_legacy_sql=false --maximum_bytes_billed=1073741824 \
'SELECT COUNT(*) AS events, COUNT(DISTINCT user_pseudo_id) AS users
```

```
  FROM `bigquery-public-data.ga4_obfuscated_sample_ecommerce.events_*`
  WHERE _TABLE_SUFFIX BETWEEN "20210101" AND "20210107"'
```

Tests to run

```
dbt debug          # asserts: profiles.yml parses, project is valid, BigQuery connection OK
dbt deps           # asserts: all four packages resolve and install into dbt_packages/
gcloud auth application-default print-access-token   # asserts: ADC token is mintable
```

The `bq` smoke query asserts that ADC can read the public dataset, that the `US` location resolves, and - critically - that pruning seven shards keeps the scan far under the 1 GiB cap passed via `--maximum_bytes_billed`. A useful sanity check: a single `events_YYYYMMDD` shard in this sample is on the order of tens of MB, so seven shards must bill far below 1 GiB; if `bq` reports it would scan gigabytes, the `_TABLE_SUFFIX` filter is not being applied (often because it was placed in a subquery the optimizer cannot prune, or the wildcard table name was mistyped).

Success criteria

- `dbt debug` prints `All checks passed!` (connection, profile, project all green).
- `gcloud auth application-default print-access-token` returns a token (ADC works).
- The smoke query returns a non-zero `events` count and bills well under 1 GiB (visible in the job stats), proving shard pruning works.
- `mcp_servers.yaml` exists as a stub; `git status` shows no `*.json` key staged.

Common mistakes

- IAM: granting `roles/owner` (or `roles/editor`) instead of least-privilege `roles/bigquery.dataViewer` + `roles/bigquery.jobUser`. Fix: bind exactly those two roles; `jobUser` is required to *run* a query, `dataViewer` only to *read* a table.
- Forgetting ADC entirely: dbt fails with "Could not automatically determine credentials." Fix: run `gcloud auth application-default login`, or set `GOOGLE_APPLICATION_CREDENTIALS` to the SA key path.
- Location mismatch: leaving `location:` unset or set to a region (`us-central1`). The GA4 public sample lives in the `US` multi-region, so any other value yields "Not found: Dataset ... in location ...". Fix: `location: US` in both `profiles.yml` targets.
- Cost blowout: running the smoke query without a `_TABLE_SUFFIX` filter scans all 90+ shards (full history). Fix: always bound `_TABLE_SUFFIX` and pass `--maximum_bytes_billed`; `profiles.yml` also sets the 5 GiB per-query cap.
- Committing the SA JSON key: leaks credentials into git history. Fix: `.gitignore` excludes `*.json`; dev should use OAuth ADC and avoid keys altogether.
- Too many threads in `profiles.yml` against a small dev quota causes job-queue thrash. Keep dev `threads: 8` as pinned.
- Omitting `maximum_bytes_billed` in `profiles.yml`: a careless later query then silently scans (and bills) the whole dataset instead of failing fast. Fix: keep the 5 GiB dev cap / 20 GiB prod ceiling from the pinned `profiles.yml`; this cap is the project's per-run byte budget and the primary cost guardrail.
- Pointing `profile:` in `dbt_project.yml` at a name that does not match the top-level key in `profiles.yml`: `dbt debug` fails with "Could not find profile named 'helios'." Fix: both must read `helios`.

M1 - Sources, Macros & Seeds

Objective

Declare the GA4 source contract and create the four shared abstractions every downstream layer reuses exactly once: the typed param extractor, the canonical session-key, the single-source-of-truth channel grouping, and the revenue reconciliation test - plus the seed that backs channel mapping. After M1 the project has no models yet, but `dbt deps` + `dbt seed` succeed and every macro compiles. These macros are load-bearing: putting channel logic or the session-key expression in more than one place is the root cause of distinct-count drift and an inventable 11th channel group.

Files to create

Path	Copy from
`models/staging/src_ga4.yml`	DBT_GUIDE.md section 3.1 (and the fuller version in section 2, "sources.yml for src_ga4") - paste verbatim; set `database` / `schema` to `bigquery-public-data` / `ga4_obfuscated_sample_ecommerce`
`macros/get_event_param.sql`	DBT_GUIDE.md section 3.4 ("get_event_param") - paste verbatim
`macros/sessionize.sql`	DBT_GUIDE.md section 3.4 ("sessionize") - paste verbatim
`macros/channel_group.sql`	DBT_GUIDE.md section 3.4 ("channel_group / channel_group_case") - paste verbatim; the file defines BOTH `channel_group()` and `channel_group_case()`
`macros/test_revenue_reconciles.sql`	DBT_GUIDE.md section 6 (custom generic test) - the `revenue_reconciles` mart-total == source-total generic test
`seeds/channel_group_mapping.csv`	canonical `source,medium,channel_group` rows (header below); feeds `channel_group_case`

The seed is a small CSV with a header row `source,medium,channel_group` and one row per known mapping; it must only ever resolve to the canonical 10 groups (`Direct, Organic Search, Paid Search, Display, Paid Social, Organic Social, Email, Affiliates, Referral, Other`). Its `+column_types` (all `string`) are already declared in `dbt_project.yml` under `seeds:` .

`src_ga4.yml` carries the source contract: `database: bigquery-public-data` , `schema: ga4_obfuscated_sample_ecommerce` , the `events` table with `identifier: 'events_*'` , the production freshness SLA (`warn_after: 36h` , `error_after: 48h`), and `not_null` tests on `event_date` , `event_timestamp` , `event_name` , `user_pseudo_id` . On the static sample, freshness is informational only - keep the `dbt_utils.recency` source test at `severity: warn` so a frozen sample does not hard-fail every build.

Commands to run

```
.venv\Scripts\activate             # POSIX: source .venv/bin/activate
dbt deps                           # (re)install packages if not already present
dbt seed                           # loads channel_group_mapping.csv into the dev dataset
dbt parse                          # compiles the project graph; surfaces macro syntax errors
# Prove each macro compiles by rendering it in a throwaway compile:
dbt compile --inline \
  "select {{ sessionize('user_pseudo_id','ga_session_id') }} as session_key"
dbt compile --inline \
  "select {{ channel_group_case('source','medium','gclid') }} as channel_group"
```

Tests to run

```
dbt seed                # asserts: the CSV loads; column types match dbt_project.yml seeds: block
dbt parse               # asserts: src_ga4.yml is valid YAML and all macros are syntactically valid Jinja+SQL
dbt compile --inline "select {{ get_event_param('ga_session_id','int') }}"   # asserts: macro renders the correlated UNNEST subquery
```

A passing `dbt parse` proves the source declaration and the macro definitions are well-formed; the inline `dbt compile` calls prove each macro renders to valid BigQuery SQL (the real golden-value unit tests on `sessionize` / `channel_group_case` land alongside the M3 keystones).

Success criteria

- `dbt deps` and `dbt seed` both exit 0; `channel_group_mapping` appears as a table in `helios_dev` .
- `dbt parse` succeeds with no compilation errors.
- `sessionize()` renders exactly `to_hex(md5(concat(user_pseudo_id, '-', cast(ga_session_id as string))))` - the one canonical expression, nowhere else.
- `channel_group_case()` renders a CASE producing exactly the 10 canonical groups and is `gclid` -aware (a non-null `gclid` forces Paid Search).
- The seed's `channel_group` column contains no value outside the 10-group set.

Common mistakes

- Pinning packages loosely or not at all: a `dbt_utils` major bump breaks `generate_surrogate_key` / `accepted_range`. Fix: keep the version ranges from `packages.yml` (e.g. `dbt_utils ≥1.1.0,<2.0.0`) and re-run `dbt deps`.
- Duplicating channel logic: writing a second `CASE` that buckets channels anywhere other than `channel_group_case()`. Fix: every model and the semantic layer must call the one macro; there is no inline channel CASE.
- Inventing an 11th group (e.g. "Paid Other") in the macro or seed. Fix: the only allowed outputs are the 10 canonical groups; anything else is a hard error in the M3/M4 `accepted_values` test.
- Registry filename drift: later wiring `mcp_servers.yaml` at `semantic_models.yml` when the live file is `semantic_layer.yaml`. Not an M1 file, but note it now - the macro/seed conventions feed that registry.
- Source filename drift: `_ga4__sources.yml` vs `src_ga4.yml`. Either name works as long as staging refs `source('src_ga4', 'events')`; pick one and keep `name: src_ga4` inside it. The pinned milestone path is `src_ga4.yml`.
- Treating freshness as a hard gate on the sample: the newest shard is `events_20210131`, so a 48h `error_after` trips forever. Fix: keep freshness informational (`severity: warn`) on dev/sample; the `prod` target enables the `dbt source freshness && dbt build` gate.
- Forgetting the seed `+column_types`: BigQuery may infer the wrong type for an all-numeric `source` value. The `dbt_project.yml` `seeds:` block already pins all three columns to `string` - do not remove it.

M2 - Staging Models

Objective

Build the renaming layer: two `view` staging models that are pure 1:1 projections of the GA4 source - rename to snake_case, cast types, parse `_TABLE_SUFFIX` into a real `event_date`, lightly flatten the scalar structs (`device.*`, `geo.*`, `ecommerce.*`), and unnest `event_params` once. Staging does no joins, no aggregations, no dedup, and no `SELECT *` against the source. After M2, `dbt build --select staging` passes with key uniqueness/not_null tests green. These two models are the only place in the entire project that touches the raw `event_params` array and the nested structs.

Files to create

Path	Copy from
`models/staging/stg_ga4__events.sql`	DBT_GUIDE.md section 3.2 - paste verbatim (1:1 typed/renamed events; `session_key` via `sessionize()`; session-scoped `event_params` source/medium plus first-touch fallback)
`models/staging/stg_ga4__event_params.sql`	DBT_GUIDE.md section 3.3 - paste verbatim (long `(event x param key)` table; the only `UNNEST(event_params)` in the repo)
`models/staging/stg_ga4__schema.yml`	DBT_GUIDE.md section 3.5 - paste verbatim (docs + `not_null` / `unique` /range tests)

Key points carried in the code you copy: `stg_ga4__events` selects an explicit column list (never `SELECT *` against the source), computes `event_date = parse_date('%Y%m%d', _table_suffix)`, surfaces `ga_session_id` and the session-scoped `event_source` / `event_medium` / `gclid` via `get_event_param( … )`, keeps `traffic_source.*` only as `first_touch_*` (FALLBACK, not session source), and emits `session_key` exclusively through `sessionize()`. The `not_null` test on `session_key` is conditioned with `where: "ga_session_id is not null"` because GA4 emits a few session-less events (e.g. `first_open`).

Commands to run

```
.venv\Scripts\activate              # POSIX: source .venv/bin/activate
dbt build --select staging          # builds BOTH views AND runs their schema.yml tests in one pass
# Inspect the materialized views / sample output:
dbt show --select stg_ga4__events --limit 5
dbt build --select +stg_ga4__event_params   # the model plus its (source) upstream, if needed
```

Tests to run

```
dbt build --select staging          # builds + tests staging in dependency order (do NOT run run+test separately)
dbt test  --select stg_ga4__events  # asserts: session_key not_null (where ga_session_id is not null),
                                    #          event_date/event_timestamp/user_pseudo_id not_null,
                                    #          purchase_revenue_in_usd ≥ 0
dbt test  --select stg_ga4__event_params   # asserts: param_key/event_timestamp/user_pseudo_id not_null
```

`dbt build --select staging` is the canonical command: it materializes the two views and runs every test attached to them in one DAG pass, so a model can never be "built but untested." Running `dbt run` then `dbt test` as separate steps risks shipping an untested staging layer.

Success criteria

- `dbt build --select staging` exits 0; both `stg_ga4__events` and `stg_ga4__event_params` exist as views in `helios_dev`.
- All `not_null` tests pass; `session_key` is non-null for every row with a non-null `ga_session_id`.
- `event_date` is a real `DATE` (not the `YYYYMMDD` string) on every row.
- No staging model contains a literal `bigquery-public-data...` reference - both read through `source('src_ga4', 'events')`.
- The only `UNNEST(event_params)` in the repo lives in `stg_ga4__event_params`.

Common mistakes

- Running `dbt run` and `dbt test` separately instead of `dbt build`: leaves a window where staging is materialized but unverified. Fix: always `dbt build --select staging`.
- `SELECT *` against the source: scans every nested column (cost) and lets a new GA4 export column silently leak into marts. Fix: enumerate every column explicitly, as the copied SQL does; the inner CTE is already explicit before the final `select *`.
- Using event-level `traffic_source.*` as the session source: it is GA4 user first-touch attribution, identical across all of a user's sessions, and mis-credits returning users. Fix: surface session-scoped `event_params.source/medium` as `event_source` / `event_medium` and keep `traffic_source.*` as `first_touch_*` fallback only (resolution happens in the M3 sessionized keystone).
- Not pruning shards: a staging view over `events_*` with no `_TABLE_SUFFIX` bound scans all shards when built. Fix: the prod build narrows via `_TABLE_SUFFIX` /lookback (see the commented incremental note in `stg_ga4__events`); on the static sample the window is the fixed sample range.
- Hand-writing the session key as `FARM_FINGERPRINT(...)` or `COUNT(*)` for sessions: causes distinct-count drift versus the canonical `TO_HEX(MD5(...))`. Fix: only ever emit `session_key` via `sessionize()`; count sessions as `COUNT(DISTINCT session_key)`.
- An unconditional `not_null` on `session_key`: fails on the legitimate session-less events. Fix: keep the `config: {where: "ga_session_id is not null"}` clause from the copied schema.yml.
- Aggregating or deduping in staging: any `GROUP BY` or filtering of business rows belongs downstream. Fix: staging stays a pure 1:1 projection; sessionization and the funnel logic are the M3 intermediate concern.
- Referencing the source by a hard-coded table name (`from \` bigquery-public-data.ga4_obfuscated_sample_ecommerce.events_*` ) instead of {{ source('src_ga4', 'events') }} : breaks lineage and the dev/prod repoint. Fix: always go through source() ; dbt_project_evaluator` flags any model that skips it.
- Building staging before M1 is green (macros not compiling, seed not loaded): `stg_ga4__events` calls `sessionize()` and `get_event_param()`, so a missing macro is a hard compile error. Fix: confirm `dbt parse` is clean and `dbt seed` has run before `dbt build --select staging`.

M3 - Sessionization & Funnel Keystone

Objective

Reconstruct the two entities GA4 never ships as rows - the **session** and the **funnel** - from the raw event stream, at one row per session (PK `session_key`). These are THE keystones: `int_ga4__sessionized` and `int_ga4__funnel_steps`. They **fail silently** - a wrong MD5 concat order still produces a valid-looking key; a non-monotonic flag still produces a number; a first-touch attribution

leak still returns rows. Nothing errors; the numbers are just quietly wrong, and every mart, metric, and agent diagnosis built on top inherits the corruption. So the rule for M3 is **golden-value tests FIRST**: write the dbt `unit_tests` and the two singular tests, watch them fail on synthetic input, then make them pass. Get the canonical `session_key` (`COUNT(DISTINCT session_key)` everywhere), the `traffic_source` first-touch FALLBACK (prefer session-scoped `event_params.source/medium` , fall back to `traffic_source` only when null), and the `reached_*` max-downstream monotonic flags exactly right.

Files to create

Path	Copy from
`models/intermediate/int_ga4__sessionized.sql`	DBT_GUIDE.md Â§4.1 (full SQL); cross-check DATA_MODEL.md Â§5.5 (grain/PK/columns) and Â§5.2-5.4 (derived attrs)
`models/intermediate/int_ga4__funnel_steps.sql`	DBT_GUIDE.md Â§4.2 (full SQL); cross-check DATA_MODEL.md Â§5.6 (max-downstream flags)
`models/intermediate/int_ga4__schema.yml`	DBT_GUIDE.md Â§4.3 (structural tests: `session_key` unique+not_null, `channel_group` accepted_values of the 10, `engaged_session` / `is_new_user` not_null, `session_revenue` range)
`models/intermediate/_int__unit_tests.yml`	DBT_GUIDE.md Â§6.5 (the 3 keystone golden-value unit tests - WRITE FIRST)
`tests/assert_funnel_monotonicity.sql`	DBT_GUIDE.md Â§6.3 (singular: returns offending sessions; 0 rows = pass)
`tests/assert_session_conversion_rate_bounds.sql`	DBT_GUIDE.md Â§6.3 (singular: conv-rate stays in plausible bounds)

Note `tests/assert_session_conversion_rate_bounds.sql` references `fct_daily_funnel` (an M4 mart), so it only goes green once M4 lands; author it now (TDD) but expect it red until then. `assert_funnel_monotonicity.sql` references `fct_funnel` (also M4) - the *intermediate-grain* monotonicity is guarded immediately by the `test_reached_flags_are_max_downstream` unit test, which runs against `int_ga4__funnel_steps` on synthetic input.

Both intermediates are materialized **`ephemeral`** (inlined as CTEs into their mart consumers - no storage, no extra scan). They are plumbing: never exposed to BI or the semantic layer directly. The semantic layer references **only marts**; marts (`fct_sessions` , `fct_funnel` , `fct_daily_funnel`) are built *from* these intermediates. Because they are internal and inlined, you cannot `dbt run --select` them in isolation and see a table - you validate them through `dbt compile` (refs/macros resolve) plus their unit/singular tests. The default is `ephemeral` ; switch a single intermediate to `view` only if several marts consume it and inlining would duplicate the scan.

Two derivation rules in `int_ga4__sessionized` are easy to get wrong and earn explicit golden tests: `is_new_user` is derived from `ga_session_number = 1` (the user's first-ever session), and `engaged_session` from the canonical rule `session_engaged = '1' OR engagement_time_msec ≥ 10000` (use `≥` , no extra clauses). `channel_group` flows from the **resolved** session-scoped source/medium (after the first-touch fallback) through `channel_group_case()` - never from the raw `traffic_source` struct.

Commands to run

```
# activate venv  (Windows: .venv\Scripts\activate ; POSIX: source .venv/bin/activate)
.venv\Scripts\activate

# build staging first (M2 dependency must be green), then the intermediates.
# ephemeral models are not built standalone - they compile into consumers, so
# you validate them via `compile` + their unit/singular tests.
dbt deps
dbt compile --select int_ga4__sessionized int_ga4__funnel_steps   # macros + refs resolve

# run the keystone unit tests (golden values) - these are the real gate
dbt build --select staging                                        # ensure upstream is green
dbt test --select int_ga4__sessionized int_ga4__funnel_steps      # unit + schema tests
# or by tag once tagged:
dbt build -s tag:keystone
```

On the static sample always reset with `dbt build --full-refresh` (3 months rebuild in seconds); the `is_incremental()` branch is production-only and never fires here.

Tests to run

```
# 1. Golden-value unit tests (synthetic input, <1s, every build) - DBT_GUIDE Â§6.5
dbt test --select int_ga4__sessionized,test_type:unit
dbt test --select int_ga4__funnel_steps,test_type:unit
#   asserts: u1+session 100 collapses 2 events → 1 session; gclid+cpc ⇒ 'Paid Search';
#            a session whose ONLY event is 'purchase' back-fills ALL upstream reached_* = true;
#            a duplicated purchase counts session_revenue ONCE.

# 2. Structural schema tests - DBT_GUIDE Â§4.3
dbt test --select int_ga4__sessionized int_ga4__funnel_steps
#   asserts: session_key unique + not_null on BOTH; channel_group in the 10 accepted values.

# 3. Singular invariants - DBT_GUIDE Â§6.3 (green after M4 marts exist)
dbt test --select assert_funnel_monotonicity assert_session_conversion_rate_bounds
#   monotonicity: 0 rows where reached_purchase > reached_add_payment_info > ... > reached_view_item
#   conv-bounds:  no day with conv_rate >1, <0.0005, or >0.20
```

Success criteria

- `session_key` is **unique** and **not_null** on both `int_ga4__sessionized` and `int_ga4__funnel_steps` (pass/fail = green dbt test).
- The monotonicity unit test passes: a purchase-only session has every `reached_*` flag = true, proving `sessions ≥ reached_view_item ≥ reached_add_to_cart ≥ reached_begin_checkout ≥ reached_add_shipping_info ≥ reached_add_payment_info ≥ reached_purchase`.
- The sessionize golden test passes: `(u1, 100)` two events collapse to one session, `landing_page` = the earliest-timestamp `page_location`, and a `gclid` + `cpc` session resolves to `Paid Search` via `channel_group_case()`.
- A returning user's session takes its `event_params` source/medium, **not** `traffic_source` (no first-touch leak).
- `channel_group` is one of exactly the 10 canonical groups; `session_revenue ≥ 0` and is deduped (one value per `transaction_id`).

Common mistakes

- **traffic_source first-touch leak (highest-value bug).** Event-level `traffic_source.*` is GA4 **user first-touch** - identical on every session that user ever has. Using it as the session source mis-credits every returning session to its original acquisition channel and corrupts every channel-level rate (Simpson's-paradox-grade). Fix: prefer session-scoped `event_params.source/medium` (surfaced as `event_source` / `event_medium` in staging) and `coalesce( … )` to `traffic_source.*` **only when the session value is null** (DATA_MODEL Â§5.3).
- **Not dropping NULL `ga_session_id`.** GA4 emits session-less events (e.g. `first_open` / `first_visit`) with null `ga_session_id`; they cannot be sessionized. Filter `where ga_session_id is not null` at the intermediate layer - otherwise they inflate counts and break `session_key` not_null (DATA_MODEL Â§5.1).
- **Wrong session_key expression.** The key is `TO_HEX(MD5(CONCAT(user_pseudo_id,'-',CAST(ga_session_id AS STRING))))`, built **only** via the `sessionize()` macro. Never `FARM_FINGERPRINT`, never `COUNT(*)`, never a different separator - any drift silently changes distinct-count and the unit test catches it. `sessions = COUNT(DISTINCT session_key)`.
- **Occurrence-based (`did_*`) flags instead of `reached_*` max-downstream.** A session that purchased via a deep link without an explicit `add_to_cart` event would show `did_purchase=1`, `did_add_to_cart=0` - a step rate > 1 and a non-monotonic funnel. Use `LOGICAL_OR(event_name IN (this stage .. purchase))` so `reached_X` is true if the session fired X **or any later stage**. The `did_*` names are retired (CLAUDE.md Â§5).
- **Not deduping `session_revenue`.** A session with two `purchase` events (export retry, multi-stream) double-counts revenue. Take `MAX(purchase_revenue_in_usd) OVER (PARTITION BY session_key, transaction_id)` then sum across distinct transactions, never `SUM` of raw event rows (DBT_GUIDE Â§4.2).
- **`landing_page` = any page, not the earliest.** Use `ARRAY_AGG(page_location IGNORE NULLS ORDER BY event_timestamp LIMIT 1)[SAFE_OFFSET(0)]` - the `page_location` of the minimum-timestamp event, not an arbitrary non-null value (DATA_MODEL Â§5.2).

- **Skipping golden tests because structural tests "pass".** `not_null` / `unique` / `accepted_values` catch shape, never correctness. The keystones earn the deepest test investment in the project - write the Â§6.5 unit tests first (DBT_GUIDE Â§4.4, Â§6.5).
- **Building before staging is green.** M3 depends on M2; a failed test on staging poisons the intermediates. Always `dbt build` (interleaves model+test in DAG order), never `dbt run` then `dbt test` separately.

M4 - Marts

Objective

Build the **consumption layer** - the only models the semantic layer (and therefore the agents) are allowed to read. Marts are **wide, denormalized** facts (each row carries its own `device_category`, `channel_group`, `country`, `is_new_user`, `event_date`) so the semantic layer slices with a single `GROUP BY` and no runtime joins, serving the <=5 GiB byte budget. There are exactly **11 marts** across three folders - **core** (session/funnel spine + conformed dims), **finance** (orders + line items), **growth** (decomposition rollups + retention). Plus one snapshot (`snap_dim_items`, SCD2 on item price/category) and two singular tests. The two hardest gates: **revenue reconciles to the cent** against raw GA4, and **`dim_channels` holds exactly 10 channel groups** - no 11th, no "Paid Other".

Files to create

Core (`models/marts/core/`):

Path	Copy from
`fct_sessions.sql`	DBT_GUIDE.md Â§5.4 (grain/PK/cols); DATA_MODEL.md Â§5.7
`fct_funnel.sql`	DBT_GUIDE.md Â§5.8 (full SQL); DATA_MODEL.md Â§5.8 - **primary session grain**
`fct_daily_funnel.sql`	DBT_GUIDE.md Â§5.9 (full SQL); DATA_MODEL.md Â§5.9 - additive counts, **no stored rates**
`dim_users.sql`	DBT_GUIDE.md Â§5.4 (dim_users); DATA_MODEL.md Â§3
`dim_items.sql`	DBT_GUIDE.md Â§5.4 (dim_items); DATA_MODEL.md Â§3
`dim_channels.sql`	DBT_GUIDE.md Â§5.11 (full SQL - the 10-row literal); DATA_MODEL.md Â§3
`dim_date.sql`	DBT_GUIDE.md Â§5.4 (dim_date); DATA_MODEL.md Â§3
`core__schema.yml`	DBT_GUIDE.md Â§5.13 (core block) + Â§6.1

Finance (`models/marts/finance/`):

Path	Copy from
`fct_orders.sql`	DBT_GUIDE.md Â§5.10 (full SQL - deduped per `transaction_id`)
`fct_order_items.sql`	DBT_GUIDE.md Â§5.5 (fct_order_items); DATA_MODEL.md Â§3
`finance__schema.yml`	DBT_GUIDE.md Â§5.13 (finance block; `test_revenue_reconciles` on `gross_revenue`)

Growth (`models/marts/growth/`):

Path	Copy from
`fct_funnel_by_dim.sql`	DBT_GUIDE.md Â§5.6/Â§5.10 catalog; DATA_MODEL.md Â§5.10 (long-format, one dim at a time)
`fct_cohorts.sql`	DBT_GUIDE.md Â§5.12 (full SQL - fixed-denominator retention)
`growth__schema.yml`	DBT_GUIDE.md Â§5.13 (growth block)

Snapshot + singular tests:

Path	Copy from
`snapshots/snap_dim_items.sql`	DBT_GUIDE.md Â§5.4 note + project `snapshots:` config (target_schema `snapshots`, tag `snapshot`); SCD2 on item `price` / `category`
`tests/assert_funnel_monotonicity.sql`	DBT_GUIDE.md Â§6.3 (authored in M3; goes green here once `fct_funnel` exists)
`tests/assert_session_conversion_rate_bounds.sql`	DBT_GUIDE.md Â§6.3 (authored in M3; goes green here once `fct_daily_funnel` exists)

The five **base grains** the semantic layer resolves against are exactly `fct_funnel`, `fct_sessions`, `fct_orders`, `fct_order_items`, `fct_cohorts`; the rest are conformed dims (`dim_*`) or pre-aggregated rollups (`fct_daily_funnel`, `fct_funnel_by_dim`) read indirectly. Materializations: the three session-grain core facts (`fct_sessions`, `fct_funnel`, `fct_daily_funnel`) are `incremental` (`insert_overwrite`, `partition_by event_date`, `cluster_by device_category, channel_group`, `require_partition_filter=true`); everything else (`dim_*`, finance, growth) is `table`.

Why marts are **wide (denormalized)** and not normalized: BigQuery is a columnar scan engine where column projection is nearly free and joins are comparatively expensive, so carrying the handful of low-cardinality dims physically on each fact costs trivial storage and removes the join from the hot path - exactly what serves the <=5 GiB/run byte budget and <5 min time-to-diagnosis. Surrogate keys (`session_key`, `order_key`, `channel_key`, `date_key`, `user_key`, `item_key`, `order_item_key`) still exist so `relationships` tests can enforce referential integrity, but the agents never need those joins at query time. `dim_channels`, `dim_date`, and `dim_users` are **conformed** - the same physical dim is shared by every fact - so a slice by `channel_group` means the same thing everywhere, which is what lets Decompose pivot any metric across the same canonical axes (DBT_GUIDE Â§5.2).

Honesty on the static sample. The incremental config (3-day `is_incremental()` lookback + `insert_overwrite` partition replacement) is the **production** design - wired and correct so the moment Helios points at a live daily GA4 export it works without code changes. The public dataset is frozen (2020-11-01..2021-01-31), so no new shards land and the `is_incremental()` branch never fires; in dev always reset with `dbt build --full-refresh` (3 months rebuild in seconds, byte-identical, idempotent).

Commands to run

```
.venv\Scripts\activate

# build order is the DAG (dbt resolves it from ref()); on the static sample reset clean:
dbt build --full-refresh        # whole DAG incl. tests, idempotent on the frozen sample

# selective builds (model + all upstream) while iterating:
dbt build --select +fct_funnel          # int keystones + fct_sessions + fct_funnel
dbt build --select +fct_daily_funnel    # also pulls fct_funnel underneath
dbt build --select +fct_orders          # fct_sessions (for wide dims) + fct_orders
dbt build --select marts.core marts.finance marts.growth

# snapshot (SCD2 history on item price/category):
dbt snapshot

# per-model tests (fast checks):
dbt test --select fct_orders            # incl. revenue_reconciles
dbt test --select dim_channels          # incl. accepted_values = the 10
```

Always `dbt build` (interleaves model + test in DAG order), never `dbt run` then `dbt test` - a separate run materializes the whole graph on bad upstream data and only discovers it afterward.

Tests to run

```
# 1. Revenue reconciles TO THE CENT (custom generic test) - DBT_GUIDE Â§6.4
dbt test --select fct_orders
#   test_revenue_reconciles on gross_revenue: |mart_total - raw_total| / raw_total ≤ tolerance.
#   fct_orders uses tolerance 0 (to the cent); fct_funnel.session_revenue uses the 0.5% default.

# 2. dim_channels = EXACTLY 10 groups - DBT_GUIDE Â§5.13 / Â§6.1
```

```
dbt test --select dim_channels
#   channel_key unique+not_null; channel_group unique+not_null; accepted_values = the 10:
#   Direct, Organic Search, Paid Search, Display, Paid Social, Organic Social,
#   Email, Affiliates, Referral, Other.  (also: SELECT count(*) FROM helios.dim_channels = 10)

# 3. PK uniqueness + referential integrity on every mart - DBT_GUIDE Â§5.13
dbt test --select marts
#   every grain PK unique+not_null; relationships FKs (fct_funnel.channel_group → dim_channels,
#   fct_orders.session_key → fct_sessions, fct_order_items.item_key → dim_items, ...).

# 4. Rollup monotonicity + retention bounds
dbt test --select fct_daily_funnel fct_cohorts assert_funnel_monotonicity assert_session_conversion_rate_bounds
#   fct_daily_funnel: purchasing_sessions ≤ sessions (expression_is_true);
#   fct_cohorts: retention_rate in [0,1] (fixed-denominator);
#   singular monotonicity returns 0 rows.

# 5. Whole-DAG green:
dbt build --full-refresh && dbt source freshness
```

Success criteria

- `dbt build` passes the whole DAG including all tests (green).
- **Revenue reconciles to the cent**: `test_revenue_reconciles` on `fct_orders.gross_revenue` passes at tolerance 0 - mart revenue equals raw `SUM(purchase_revenue_in_usd)` over `event_name='purchase'`. `fct_funnel.session_revenue` reconciles within 0.5%, and session-grain `revenue` equals order-grain `gross_revenue` at the grand total.
- `dim_channels` **= exactly 10 rows**, one per canonical group; `accepted_values` passes with no 11th group.
- Every mart's grain PK is unique + not_null; every conformed FK passes its `relationships` test.
- `fct_daily_funnel` stores **only additive counts** (no rates) and satisfies `purchasing_sessions ≤ sessions`; rates are derived later in the semantic layer as `SUM(num)/SUM(den)`.
- `fct_daily_funnel` aggregates `fct_funnel` (which carries `session_revenue`), not `int_ga4__funnel_steps` - revenue is present at the rollup grain.
- `fct_cohorts.retention_rate` is in `[0,1]` and monotonically non-increasing (fixed original-cohort denominator).

Common mistakes

- **Revenue not deduped per** `transaction_id`**.** GA4 emits retry/multi-stream duplicate `purchase` rows for one order. `fct_orders` must collapse to one row per `transaction_id` (e.g. `GROUP BY order_key` with `ANY_VALUE(...)`); failing to dedup double-counts revenue and AOV and breaks `test_revenue_reconciles` (DBT_GUIDE Â§5.10).
- **Aggregating non-** `_in_usd` **columns or including shipping/tax in revenue.** Use only the `_in_usd` twins (`purchase_revenue_in_usd`); never the native-currency columns. Revenue excludes shipping and tax (those are separate columns: `shipping_value_in_usd`, `tax_value_in_usd`); `net_revenue = gross_revenue - refund_value_in_usd` (CLAUDE.md Â§5, DBT_GUIDE Â§5.10).
- **Session-grain** `revenue` **not reconciling to order-grain** `gross_revenue`**.** `fct_funnel.session_revenue` (deduped per transaction) and `fct_orders.gross_revenue` (deduped per transaction) must tie at the grand total. If they diverge, one path is double-counting or dropping untagged purchases (`ecommerce.transaction_id IS NULL` is excluded in `fct_orders`).
- **Inventing an 11th channel group.** `dim_channels` enumerates **exactly 10** rows; there is no "Paid Other". Do not duplicate channel logic anywhere - it lives only in `channel_group_case()` (`macros/channel_group.sql`); `dim_channels` just enumerates the same 10 strings as the conformed dimension (DBT_GUIDE Â§5.11). A stale incremental partition can drift the mart from the macro, which is why `accepted_values` is tested at the *mart* the agents read, not just the macro (DBT_GUIDE Â§6.1).
- **Storing rates in** `fct_daily_funnel`**.** Store **only additive counts** (`sessions`, `view_item_sessions` â€¦ `purchasing_sessions`, `transactions`, `revenue`). Rates (`session_conversion_rate`, step rates) are derived in `semantic_layer.yaml` as `SUM(num)/SUM(den)` after grouping. Pre-dividing breaks re-aggregation across slices and reintroduces Simpson's paradox (DBT_GUIDE Â§5.9, DATA_MODEL Â§5.9).
- **Averaging per-segment ratios.** Never average pre-divided segment rates to get an aggregate. Always `SUM(numerator)/SUM(denominator)` after grouping - averaging ratios is a textbook Simpson's-paradox artifact (CLAUDE.md Â§5).

- **`fct_daily_funnel` aggregating the wrong upstream.** It must aggregate `fct_funnel` (carries `session_revenue`), not `int_ga4__funnel_steps`. Dropping revenue here breaks the eval's dollar-at-risk labels (CLAUDE.md Â§8, DBT_GUIDE Â§5.9).
- **`SELECT *` over `events_*` / no `maximum_bytes_billed` / scanning all 90+ shards.** Prune via `_TABLE_SUFFIX` (the sample window is `'20201101'..'20210131'`); the incremental config's `require_partition_filter=true` makes a date-less query error rather than full-scan. Name every column explicitly.
- **Running `dbt run` then `dbt test` separately, or referencing ephemeral models in singular tests.** `dbt build` interleaves model + test in DAG order so a poisoned upstream aborts before reaching the marts. Singular tests must reference the *materialized* marts (`fct_funnel`, `fct_daily_funnel`), not the ephemeral `int_ga4__*` (which inline as CTEs and cannot be selected from in a standalone test).

M5 - Semantic Layer live

Objective

Make the metric registry the single source of truth and put a hard compile gate in front of it. The registry (`models/semantic/semantic_layer.yaml`, registry v2.0.0 - 47 metrics / 19 dimensions / 7 entities / 4 time grains) ALREADY EXISTS and is referential-integrity validated; you do NOT regenerate it. You add (1) a thin dbt schema wrapper so the layer is part of the dbt DAG and documented, and (2) a standalone Python validator that proves every `numerator` / `denominator`, every derived `expr` `{token}`, every `dimensions_supported` entry, and every `agents` entry resolves against the live registry. The validator is the M5 exit gate and the CI compile gate referenced by `METRIC_GOVERNANCE_GUIDE.md Â§6.1` (invariant I1: no metric exists outside the registry).

Files to create

Path	Source to copy from
`models/semantic/semantic_layer.yaml`	ALREADY EXISTS - do NOT regenerate. This is the keystone (47 metrics).
`models/semantic/metrics__schema.yml`	New thin wrapper - template below. Spec: `METRIC_GOVERNANCE_GUIDE.md Â§2`, `CLAUDE.md Â§5`.
`scripts/validate_semantic.py`	New - FULL code below (the reader has no AI; copy it verbatim). Spec: `MCP_ARCHITECTURE.md Â§7-Â§8`, `METRIC_GOVERNANCE_GUIDE.md Â§6.1`.

`metrics__schema.yml` is a documentation/exposure shell - the semantic layer materializes as a `view` (per `CLAUDE.md Â§5`) and the YAML registry is the governance object. Create it as:

```yaml
# models/semantic/metrics__schema.yml
version: 2
exposures:
  - name: semantic_layer
    type: application
    maturity: high
    owner: {name: analytics-eng, email: analytics-eng@helios}
    description: >
      Helios semantic registry (semantic_layer.yaml, v2.0.0). The single source of
      truth for all 47 metrics / 19 dimensions. semantic-mcp.build_query reads it;
      every governed SQL string derives from it. Referential integrity is enforced
      by scripts/validate_semantic.py (CI compile gate).
    depends_on:
      - ref('fct_funnel')
      - ref('fct_sessions')
      - ref('fct_orders')
      - ref('fct_order_items')
      - ref('fct_cohorts')
```

Now `scripts/validate_semantic.py` - copy verbatim:

```python
#!/usr/bin/env python3
"""Referential-integrity compile gate for the Helios semantic registry.

Loads models/semantic/semantic_layer.yaml and asserts, with ZERO dangling refs:
  1. every metric's numerator/denominator (ratio) resolves to a defined metric_name
  2. every derived metric's expr {token} resolves to a defined metric_name
  3. every dimensions_supported entry resolves to a defined dimension_name
  4. every agents[] entry is one of the 6 valid metric-consuming agents
  5. every metric's grain is a defined grains: key
  6. the type -> population matrix holds (ratio has num+den; derived has expr)
  7. all canonically-required metric names are present
Exits 1 (and prints FAIL with every error) if any check fails; prints PASS otherwise.
No AI, no network, no warehouse - pure file validation.
"""
import re
import sys
from pathlib import Path

try:
    import yaml
except ImportError:
    sys.exit("FAIL: pyyaml not installed. Run: pip install pyyaml")

REGISTRY = Path(__file__).resolve().parents[1] / "models" / "semantic" / "semantic_layer.yaml"

# The 6 metric-consuming agents (Orchestrator plans but consumes no metrics).
VALID_AGENTS = {"Monitor", "Decompose", "Diagnose", "Prescribe", "Narrator", "Critic"}

# A floor of metric names that MUST exist (headline + funnel + decomposition spine).
REQUIRED_METRICS = {
    "sessions", "users", "new_users", "returning_users", "revenue", "orders",
    "gross_revenue", "net_revenue", "aov", "revenue_per_session", "revenue_per_user",
    "view_item_sessions", "add_to_cart_sessions", "begin_checkout_sessions",
    "purchasing_sessions", "session_conversion_rate", "user_conversion_rate",
    "view_to_cart_rate", "cart_to_checkout_rate", "checkout_to_purchase_rate",
    "traffic_share", "channel_revenue", "channel_conversion_rate",
}

EXPR_TOKEN = re.compile(r"\{([a-zA-Z0-9_]+)\}")  # matches {revenue}, {sessions}, ...


def load_registry(path: Path) -> dict:
    if not path.exists():
        sys.exit(f"FAIL: registry not found at {path}")
    with path.open("r", encoding="utf-8") as fh:
        return yaml.safe_load(fh)


def validate(reg: dict) -> list[str]:
    errors: list[str] = []

    metrics = reg.get("metrics") or []
    dims = reg.get("dimensions") or []
    grains = reg.get("grains") or {}

    metric_names = {m["metric_name"] for m in metrics}
    dim_names = {d["dimension_name"] for d in dims}
    grain_keys = set(grains.keys())

    # Sanity: registry shape.
    if not metric_names:
        errors.append("registry has no metrics:")
    if not dim_names:
        errors.append("registry has no dimensions:")

    for m in metrics:
        name = m.get("metric_name", "<unnamed>")
        mtype = m.get("type")

        # (5) grain must resolve
        grain = m.get("grain")
```

```
        if grain not in grain_keys:
            errors.append(f"{name}: grain '{grain}' not in grains: {sorted(grain_keys)}")

        # (1) ratio numerator/denominator must resolve to defined metrics
        if mtype == "ratio":
            for slot in ("numerator", "denominator"):
                ref = m.get(slot)
                if ref is None:
                    errors.append(f"{name}: type=ratio but {slot} is null")
                elif ref not in metric_names:
                    errors.append(f"{name}: {slot} '{ref}' is not a defined metric_name")
            if m.get("expr") is not None:
                errors.append(f"{name}: type=ratio must leave expr null")

        # (2) derived expr {token}s must resolve to defined metrics
        elif mtype == "derived":
            expr = m.get("expr")
            if not expr:
                errors.append(f"{name}: type=derived but expr is null/empty")
            else:
                tokens = EXPR_TOKEN.findall(expr)
                if not tokens:
                    errors.append(f"{name}: derived expr '{expr}' has no {{token}}")
                for tok in tokens:
                    if tok not in metric_names:
                        errors.append(f"{name}: expr token '{{{tok}}}' is not a defined metric_name")

        # type → population matrix for count/sum
        elif mtype in ("count", "sum"):
            if not m.get("sql_definition"):
                errors.append(f"{name}: type={mtype} requires sql_definition")
            for slot in ("numerator", "denominator", "expr"):
                if m.get(slot) is not None:
                    errors.append(f"{name}: type={mtype} must leave {slot} null")
        else:
            errors.append(f"{name}: unknown type '{mtype}' (expected count|sum|ratio|derived)")

        # (3) dimensions_supported must resolve to defined dimensions
        for d in (m.get("dimensions_supported") or []):
            if d not in dim_names:
                errors.append(f"{name}: dimensions_supported '{d}' is not a defined dimension_name")

        # (4) agents must be valid
        for a in (m.get("agents") or []):
            if a not in VALID_AGENTS:
                errors.append(f"{name}: agent '{a}' not in {sorted(VALID_AGENTS)}")

    # (6) every required canonical metric is present
    for req in sorted(REQUIRED_METRICS):
        if req not in metric_names:
            errors.append(f"required metric '{req}' missing from registry")

    return errors


def main() -> int:
    reg = load_registry(REGISTRY)
    metrics = reg.get("metrics") or []
    dims = reg.get("dimensions") or []
    errors = validate(reg)
    if errors:
        print(f"FAIL: {len(errors)} dangling/invalid reference(s) in {REGISTRY.name}:")
        for e in errors:
            print(f"  - {e}")
        return 1
    print(
        f"PASS: {len(metrics)} metrics, {len(dims)} dimensions - "
        f"0 dangling references (numerator/denominator/expr/dimensions/agents/grain all resolve)."
    )
    return 0
```

```
if __name__ == "__main__":
    sys.exit(main())
```

Commands to run

```
# from repo root, venv active (POSIX: source .venv/bin/activate | Windows: .venv\Scripts\activate)
pip install pyyaml                        # if not already from requirements.txt
python scripts/validate_semantic.py       # the M5 gate
dbt parse                                 # registers the exposure; no warehouse needed
dbt docs generate                         # optional: surfaces the semantic exposure in docs
```

Tests to run

```
# 1. The gate must print PASS and exit 0 on the shipped registry:
python scripts/validate_semantic.py ; echo "exit=$?"
# asserts: 47 metrics / 19 dimensions, 0 dangling references.

# 2. Negative test - prove it FAILS loud on a broken ref (work on a COPY):
cp models/semantic/semantic_layer.yaml /tmp/bad.yaml
# hand-edit /tmp/bad.yaml: change one ratio denominator to 'sessionz' (typo)
# point the script at it temporarily, OR set REGISTRY env-style by copying over a scratch repo;
# expected: "FAIL: 1 dangling/invalid reference(s)" + "denominator 'sessionz' is not a defined metric_name", exit 1.
```

Success criteria

- `python scripts/validate_semantic.py` prints `PASS: 47 metrics, 19 dimensions - 0 dangling references` and exits 0.
- A deliberately broken numerator/denominator/`{token}`/dimension prints `FAIL` naming the exact offending reference and exits 1 (gate is real, not cosmetic).
- `dbt parse` succeeds with the `semantic_layer` exposure depending on the five mart grains.

Common mistakes

- Registry FILENAME drift: the registry is `semantic_layer.yaml` (v2.0.0, 47 metrics), NOT the retired `semantic_models.yml` (v1, 28 metrics). `MCP_ARCHITECTURE.md Â§3` and `CLAUDE.md Â§5` still cite the old name in places - the live file and `mcp_servers.yaml` MUST both point at `semantic_layer.yaml`. Fix: standardize on `semantic_layer.yaml` everywhere.
- Regenerating the registry: it is the keystone and is already RI-validated. Editing it to "fix" a validator error is backwards - fix the validator/typo, never silently rewrite a governed definition.
- Validating `expr` with a naive split on commas instead of extracting `{token}`s: `SAFE_DIVIDE({revenue},{users})` has two tokens, both must resolve; a substring match misses them. Use the `{ … }` regex.
- Treating an unknown agent (e.g. `Orchestrator` in a metric's `agents[]`) as fine - the Orchestrator plans but consumes no metrics; only the 6 in `VALID_AGENTS` are legal.
- Skipping the negative test, so you ship a gate that always prints PASS (it must actually exit 1 on a dangling ref, or it is not a gate).
- Forgetting the type->population matrix: a `ratio` with a stray `expr`, or a `derived` with `numerator`/`denominator` set, is malformed even if every name resolves.

M6 - Grounding MCP Pair

Objective

Build the two servers that physically enforce grounding: `semantic-mcp` (the ONLY path to SQL) and `warehouse-mcp` (the sole BigQuery client, enforcing the dry-run + byte-budget gate). When this milestone is green, an agent can name a metric and get governed SQL that is cost-checked, executed under budget, and reconciled - with zero hand-authored SQL. This is the L1 grounding spine.

Files to create

Path	Source to copy from
`helios/mcp/__init__.py`	empty package marker
`helios/mcp/base.py`	`MCP_ARCHITECTURE.md Â§4` (layout), Â§5 (error taxonomy), Â§8 (audit wrapper). Server factory + typed errors + `_h(sql)` hash + config loader.
`helios/mcp/schemas.py`	`MCP_ARCHITECTURE.md Â§6` - pydantic/JSON-Schema I/O for every tool.
`helios/mcp/semantic.py`	`MCP_ARCHITECTURE.md Â§6.2` + Â§9 resolver skeleton; binding rules Â§7. Loads `semantic_layer.yaml`, runs the Â§8 RI compile at startup, serves `get_metric` / `list_dimensions` / `build_query`.
`helios/mcp/warehouse.py`	`MCP_ARCHITECTURE.md Â§6.1` + Â§9 skeleton (copy verbatim). `dry_run` / `run_query` / `reconcile` / `list_tables` / `describe_table`.
`mcp_servers.yaml`	`MCP_ARCHITECTURE.md Â§3` - FINALIZED. Set `semantic-mcp.config.registry: ./models/semantic/semantic_layer.yaml` (NOT semantic_models.yml). Byte budget `5368709120`; `require_dry_run: true`.

Error taxonomy for `base.py` (closed set, JSON-RPC codes from Â§5) - define these as exception classes that carry `code`:

```
# helios/mcp/base.py (essentials)
import hashlib
class MCPError(Exception):
    code = -32000
    def __init__(self, message): super().__init__(message); self.message = message
class UnknownMetric(MCPError):        code = -32001
class UnknownDimension(MCPError):     code = -32002
class DimensionNotPermitted(MCPError):code = -32003
class InvalidFilter(MCPError):        code = -32004
class NotDryRunFirst(MCPError):       code = -32011
class ByteBudgetExceeded(MCPError):   code = -32012

def normalize_hash(sql: str) -> str:                # I1 dry-run gate key
    return hashlib.sha256(" ".join(sql.split()).lower().encode()).hexdigest()
```

`warehouse.py` is the load-bearing gate - copy the `MCP_ARCHITECTURE.md Â§9` skeleton verbatim: `dry_run` records `normalize_hash(sql)` into `_SEEN_DRYRUN`; `run_query` raises `NotDryRunFirst` unless the hash was seen (I1) and caps `maximum_bytes_billed = min(arg, 5_368_709_120)` (I2). `semantic.py` copies the Â§9 `build_query` resolver: `_measure()` emits `agg(sql) AS name` for count/sum, `SAFE_DIVIDE(SUM(num),SUM(den)) AS name` for ratio, `expand_expr(expr) AS name` for derived; it validates every dim against the metric's `dimensions_supported` (DimensionNotPermitted) and every name against the registry (UnknownMetric/UnknownDimension). Remember the resolver maps authoring fields to short keys (`metric_name→name`, `sql_definition→sql`, `aggregation_method→agg`, `dimensions_supported→dimensions`) per `METRIC_GOVERNANCE_GUIDE.md Â§3`.

Commands to run

```
pip install mcp google-cloud-bigquery pydantic pyyaml
# auth for warehouse-mcp (least-privilege SA, ADC):
gcloud auth application-default login
export GOOGLE_APPLICATION_CREDENTIALS=/path/to/helios-wh-key.json   # Windows: $env:GOOGLE_APPLICATION_CREDENTIALS=...
export HELIOS_WH_TOKEN=<bearer>

# RI compile gate runs at semantic-mcp startup; smoke each server:
python -m helios.mcp.semantic        # stdio - should load registry, RI-compile, then idle on stdin
python -m helios.mcp.warehouse       # streamable-http on :8443
```

Tests to run

```
pytest helios/mcp/tests -k "semantic or warehouse" -v
```

- semantic-mcp (a) unknown metric/dimension -> `UnknownMetric` / `UnknownDimension`, NO SQL emitted; (b) `build_query('session_conversion_rate',['device_category'],window='last_28d')` produces the session-keyed subquery form (snapshot match to `MCP_ARCHITECTURE.md Â§6.2`); (c) a registry with a dangling ref -> server REFUSES to start; (d) a non-whitelisted dim -> `DimensionNotPermitted`.
- warehouse-mcp guardrail chain: (a) `run_query` without a prior `dry_run` -> `NotDryRunFirst`; (b) `max_bytes_billed` capped at the 5 GiB budget, over-scan -> `ByteBudgetExceeded`; (c) `reconcile(metric, grain)` vs a hand-written control query within 0.5%; (d) a write SQL -> error (SA is read-only).
- Round-trip: `build_query → dry_run → run_query → reconcile` returns rows and reconciles to <=0.5%.

Success criteria

- `build_query → dry_run → run_query → reconcile` round-trips and reconciles within 0.5%.
- `run_query` refuses any SQL not dry-run'd this run (`NotDryRunFirst`); byte budget caps at 5 GiB (`ByteBudgetExceeded` on over-scan).
- An unknown metric is a HARD error (`UnknownMetric`), never a fallback to free SQL.
- semantic-mcp refuses to start if the registry has a dangling reference (fail loud).

Common mistakes

- Letting the LLM author raw SQL: the model passes only string names; physical columns live exclusively in registry `sql` fields. The only path to data is `build_query → dry_run → run_query`. No agent holds a raw-SQL tool.
- Calling `run_query` without a prior `dry_run`, or not capping bytes: both invariants (I1 sha256 gate, I2 byte cap) are preconditions enforced in `warehouse.py`, not agent goodwill. The hash must be normalized identically in `dry_run` and `run_query` (whitespace-collapsed, lowercased) or the gate spuriously fails.
- Registry filename drift in `mcp_servers.yaml`: `semantic-mcp.config.registry` MUST be `./models/semantic/semantic_layer.yaml`. Pointing at the retired `semantic_models.yml` loads the wrong (28-metric) layer.
- Treating an unknown metric name as a fallback to free SQL instead of a hard error (rule G5). The branch hard-stops and the agent re-plans against `list_dimensions()` /the registry.
- Giving warehouse credentials to the stdio servers: only `warehouse-mcp` (and later `report-mcp` for memory) touch credentials. semantic/stats/experiment run credential-free over stdio (least privilege).
- SELECT * over `events_*` or scanning all 90+ shards: governed SQL queries the marts (`helios.marts.fct_*`), not raw events; prune with the window predicate, and let `dry_run` reject over-budget scope so the agent narrows dims/window.
- Averaging per-segment ratios: the resolver emits `SAFE_DIVIDE(SUM(num),SUM(den))` AFTER grouping for ratios - never `AVG(per_row_ratio)` (Simpson's-paradox defense).

M6b - Stats / Experiment / Report MCP

Objective

Build the three data-independent servers that complete the L1 toolset: `stats-mcp` (the ONLY path to math - seeded, deterministic), `experiment-mcp` (powered test cards, no data access), and `report-mcp` core (`render_brief` / `export`). The keystone here is `decompose_change`, which dissolves Simpson's paradox by splitting `Î”R` into mix/rate/interaction - it fails silently if wrong, so it gets a golden-value test FIRST.

Files to create

Path	Source to copy from
`helios/mcp/stats.py`	`MCP_ARCHITECTURE.md Â§6.3` + Â§9 `decompose_change` skeleton (copy verbatim). Seeded `rng_seed: 1729`. `detect_anomaly` / `decompose_change` / `significance_test` / `forecast` / `cohort_retention` / `rfm_segment`.

Path	Source to copy from
`helios/mcp/experiment.py`	`MCP_ARCHITECTURE.md Â§6.4` . `power_analysis` / `runtime_estimate` / `design_experiment` . Validates target metric against the registry (no data access).
`helios/mcp/report.py`	`MCP_ARCHITECTURE.md Â§6.5` core only: `render_brief` / `export` . (Memory tools `save_diagnosis` / `recall_prior` come in M8 - do NOT build them here; report-mcp gets NO BigQuery client.)

Copy the `decompose_change` skeleton from `MCP_ARCHITECTURE.md Â§9` verbatim - it implements the FOUNDATION identity exactly (`mix=Î£ Î”wÂ·r0` , `rate=Î£ w0Â·Î”r` , `interaction=Î£ Î”wÂ·Î”r` , `Î”R=mix+rate+interaction`) and raises `SegmentMismatch` when `t0` / `t1` segment sets differ. `experiment.py` 's `design_experiment` validates `metric` (and guardrail metrics) against the registry so a test card can only target governed metrics. `report.py` 's `render_brief([])` raises `EmptyFindings` (-32040); it has NO warehouse client (least-privilege - rendering is pure string work).

The Â§6.2 golden example for the `decompose_change` test (two segments, weights shift, rates flat):

```
# helios/mcp/tests/test_decompose_golden.py
from helios.mcp.stats import decompose_change

def test_decompose_change_golden():
    # Â§6.2 example: weight moves desktop→mobile, both rates UNCHANGED.
    t0 = [{"seg": "desktop", "w": 0.40, "r": 0.030}, {"seg": "mobile", "w": 0.60, "r": 0.012}]
    t1 = [{"seg": "desktop", "w": 0.30, "r": 0.030}, {"seg": "mobile", "w": 0.70, "r": 0.012}]
    out = decompose_change("session_conversion_rate", "device_category", t0, t1)
    assert abs(out["mix_effect"]   - (-0.0018)) < 1e-9   # Î£ Î”wÂ·r0 = (-0.10)(0.030)+(0.10)(0.012)
    assert abs(out["rate_effect"]  - 0.0)       < 1e-9   # rates unchanged
    assert abs(out["interaction"]  - 0.0)       < 1e-9   # no Î”r, so Î”wÂ·Î”r = 0
    assert abs(out["delta_R"]      - (-0.0018)) < 1e-9   # = mix + rate + interaction
```

`mix = (0.30−0.40)*0.030 + (0.70−0.60)*0.012 = −0.0030 + 0.0012 = −0.0018` ; rate and interaction are 0 because no segment rate moved. This is the canonical "the blended rate fell purely because traffic shifted to lower-converting mobile" result - mix-shift, not behavior change.

Commands to run

```
pip install scipy statsmodels prophet pandas mcp pyyaml   # pmdarima as a prophet fallback
# smoke each stdio server:
python -m helios.mcp.stats
python -m helios.mcp.experiment
python -m helios.mcp.report
```

Tests to run

```
pytest helios/mcp/tests -v
```

- stats-mcp GOLDEN: `decompose_change` on the Â§6.2 example -> `mix_effect=-0.0018, rate_effect=0, interaction=0, delta_R=-0.0018` (Â±1e-9). Identity `mix+rate+interaction == delta_R` holds on random inputs. `significance_test` matches a scipy reference. Determinism: same seed (1729) -> byte-identical outputs.
- experiment-mcp: `power_analysis(baseline, mde, alpha=0.05, power=0.8)` two-proportion sample size matches the closed-form reference; `design_experiment` REJECTS an ungoverned metric (`UnknownMetric`).
- report-mcp: `render_brief([])` -> `EmptyFindings` ; a brief's every number traces to a backing tool-output hash (faithfulness rule).

Success criteria

- `decompose_change` golden test passes exactly: `mix=-0.0018, rate=0, interaction=0` on the Â§6.2 example.
- `power_analysis` matches the closed-form two-proportion sample size.
- `render_brief([])` raises `EmptyFindings` (and the loop emits a "no material change" brief).

- `design_experiment` cannot target a metric absent from the registry.
- Re-running any stats tool on the same input is byte-identical (seed 1729).

Common mistakes

- Computing statistics in prose instead of via stats-mcp: anomaly scores, decompositions, significance, power, forecasts are tool outputs verbatim (rule G2). The LLM never does arithmetic; numbers in the brief come from `stats` / `experiment` outputs.
- Getting `decompose_change` subtly wrong - it fails SILENTLY (wrong numbers, not an error). Use the Â§9 formulas exactly: `mix` uses `r(t0)`, `rate` uses `w(t0)`; swapping the `t0` / `t1` baselines flips the attribution. The golden test is the only thing that catches this.
- Not seeding the RNG (`rng_seed: 1729`): non-determinism breaks the byte-identical test and makes eval grading impossible. Seed scipy/statsmodels/prophet sampling at import.
- Giving report-mcp a BigQuery client: least-privilege violation. report-mcp core renders strings only; memory I/O (with the `helios_memory` write scope) is a separate M8 concern.
- Skipping the `render_brief([])` -> `EmptyFindings` path: a clean run must still produce a "no material change" brief, not crash or fabricate a finding.
- `design_experiment` not validating its target/guardrail metrics against the registry: a test card could otherwise target a non-existent metric (e.g. `refund_rate`), reintroducing ungoverned names. Validate against `semantic_layer.yaml`.
- Treating `SegmentMismatch` as ignorable: if `t0` / `t1` segment sets differ, the decomposition is undefined - align the segment sets (or the mix term double-counts appearing/disappearing segments).

M7 â€" Minimal Autonomous Loop (L1 EXIT)

Objective

Stand up the agent framework (`A8.0`) and the smallest end-to-end run: a deterministic Python FSM that goes `PLAN â†' MONITOR â†' NARRATE`, invokes the **Monitor** (Sonnet) and **Narrator** (Sonnet, `render_brief` only) agents as model-driven nodes, and emits a Decision Brief. The framework is the single enforcement point â€" it owns the per-agent tool allow-list, the `dry_run`-before-`run_query` gate (G3), the byte budget, and the audit log. This is the L1 exit: **one injected anomaly produces a brief in under 5 minutes with zero hallucinated columns** (every SQL string in the audit log originated from `semantic.build_query`). The LLM never controls flow and never authors SQL (AGENT_ARCHITECTURE Â§1, Â§13).

Files to create

Path	Copy/spec from
`helios/agents/config.py`	AGENT_ARCHITECTURE Â§9 (the tunables: `MAX_REFUTE_ROUNDS=2`, `MAX_BRANCHING=4`, `MAX_DEPTH=3`, `MIN_RATE_EFFECT=0.3pp`, `MIN_SEGMENT_SESSIONS=500`, `SIGNIFICANCE_ALPHA=0.05`, `ANOMALY_SCORE_THRESHOLD=3.0`, `PRIOR_HALF_LIFE_DAYS=60`, `BYTE_BUDGET=5 GiB`, `TIME_BUDGET_MIN=5`, `MODELS` map, `TEMPERATURE=0.0`, `RNG_SEED=1729`)
`helios/agents/framework.py`	AGENT_ARCHITECTURE Â§4 â€" `AgentSpec` dataclass (Â§4.1), the tool-call wrapper (Â§4.2: allow-list assert â†' `run_query` â‡' `dry_run_seen` assert â†' byte-budget enforce â†' retries â†' `append_audit_log`), the `Finding` schema + validator (Â§4.3), context compaction (Â§4.4)
`helios/agents/monitor.py`	AGENT_ARCHITECTURE Â§6.2 â€" `MONITOR_SPEC` (tools: `semantic.get_metric`, `semantic.build_query`, `warehouse.dry_run`, `warehouse.run_query`, `stats.detect_anomaly`, `stats.forecast`) + system prompt sketch
`helios/agents/narrator.py`	AGENT_ARCHITECTURE Â§6.7 â€" `NARRATOR_SPEC`; **M7 variant: `render_brief` only** (no `save_diagnosis` until M8). Tools at M7: `report.render_brief`, `semantic.get_metric`
`helios/runner.py`	AGENT_ARCHITECTURE Â§5 (FSM diagram + Â§5.1 transition table) + Â§10 (run trace). M7 path = the trivial `PLANâ†'MONITORâ†'NARRATE` subset
`helios/agents/tests/fixtures/`	Recorded MCP tool fixtures (canned `build_query` / `run_query` / `detect_anomaly` responses) per AGENT_ARCHITECTURE Â§12

Note: `models/semantic/semantic_layer.yaml`, `eval/scenarios/*.yaml`, and `docs/CLAUDE.md` **already exist â€" do not regenerate**. The MCP servers (`helios/mcp/*.py`) come from M6/M6b and must be running/importable before M7.

Commands to run

```
# Windows: .venv\Scripts\activate   |   POSIX: source .venv/bin/activate
source .venv/bin/activate
export ANTHROPIC_API_KEY=...           # Windows: $env:ANTHROPIC_API_KEY="..."
export GOOGLE_APPLICATION_CREDENTIALS=./helios-wh-sa.json
export HELIOS_WH_TOKEN=...

# 1) framework + node unit tests with recorded fixtures (no live BigQuery)
pytest helios/agents/tests -q

# 2) one real minimal run over a window with a known dip (clean-run also valid)
python -m helios.runner --t0 2020-12-01 --t1 2020-12-20 --window-t1b 2021-01-10 \
  --metrics session_conversion_rate --budget-gib 5 --time-budget-min 5

# 3) prove grounding: every sql_text in the audit log came from semantic.build_query
python -m helios.runner --audit-check --run-id <last_run_id>
```

Tests to run

```
pytest helios/agents/tests/test_framework.py -q   # allow-list + dry_run gate + audit
pytest helios/agents/tests/test_monitor.py -q      # returns anomaly list envelope from fixtures
pytest helios/agents/tests/test_fsm.py -q          # PLANâ†'MONITORâ†'NARRATE + clean-run short-circuit
```

- `test_framework.py`: asserts a tool **outside** the spec raises `AllowListViolation`; calling `run_query` without a prior `dry_run` on the same normalized SQL raises `NotDryRunFirst`; every call appends exactly one `audit_log` row.
- `test_monitor.py`: feeding the recorded daily-series fixture yields an anomaly-list envelope `[{metric, dim, t0, t1, score, direction}]` validated against `output_schema`.
- `test_fsm.py`: a zero-anomaly Monitor envelope routes straight to `NARRATE` (no-finding brief) â†' `END`; a one-anomaly envelope still completes (Decompose/Diagnose are stubs at M7 â€" route MONITORâ†'NARRATE directly).

Success criteria

- `pytest helios/agents/tests` is all-green.
- A single injected/real anomaly produces a rendered Decision Brief in **< 5 min** wall-clock (`run_state.ended_at â^' started_at`).
- `--audit-check` reports **0 hand-authored SQL**: every `audit_log.sql_text` is non-NULL only for `semantic-mcp.build_query`-sourced queries and **0 hallucinated columns/metrics** (every metric name resolved by `semantic-mcp`).
- The clean-run short-circuit works: zero anomalies still writes a `run_state` row and a "no material change" brief (absence of finding is recorded).

Common mistakes

- **Making the LLM the controller.** The runner is plain Python; the agent only composes tool calls and returns a typed `Finding[]`/anomaly envelope. If the model decides transitions, you have lost auditability and determinism (AGENT_ARCHITECTURE Â§1).
- **Allow-list as advice, not enforcement.** The Â§4.2 wrapper must `assert tool in agent.allowed_tools` for *every* call. A prompt that "asks nicely" is not enforcement â€" structural beats behavioral (Â§7).
- **Skipping the dry_run gate.** Calling `run_query` without a prior `dry_run` on the same SQL must raise `NotDryRunFirst` (G3). Do not retry an over-budget `dry_run` blindly â€" narrow window/dims first (Â§11).
- **Dumping row-level results into the LLM context.** Reduce `run_query` output to aggregates / `(slice, rate, p, n)` tuples before it enters the model window (Â§4.4); otherwise token cost explodes as the tree widens.
- **Expecting byte-identical LLM output.** Run at `temperature 0` but grade *statistically* via the eval harness (M10), not by string-equality. Seed only the stats (`RNG_SEED=1729`); the audit log makes the run reconstructable.

- **Building M7 before M6/M6b are green.** Monitor needs live `semantic.build_query`, `warehouse.dry_run/run_query`, and `stats.detect_anomaly`. Wire them first.

M8 â€" Memory Store (L2)

Objective

Make Helios *learn across runs*. Create the eight `helios_memory` BigQuery tables (the durable system of record), a vector store for embedding-based similarity recall, seed the calendars/glossary, and implement the `report-mcp` memory tools (`save_diagnosis` / `recall_prior`, `A7.4`) so agents read/write memory **only** through `report-mcp` (the LLM never writes memory directly). This is the prerequisite for the full Critic refutation battery (it needs the seasonality/launch priors) and for the Narrator's `save_diagnosis`. Exit: `save_diagnosis â†' recall_prior` round-trips, and the seasonality calendar is seeded with the dataset window's known events (Bible Â§22).

Files to create

Path	Copy/spec from
`sql/helios_memory_ddl.sql`	Bible Â§22.1â€"Â§22.5 â€" the 8 tables: `diagnosis_history`, `suppression_list`, `glossary`, `seasonality_calendar`, `launch_calendar`, `action_tracking`, `run_state`, `audit_log` (copy the `CREATE TABLE IF NOT EXISTS` statements verbatim, including `PARTITION BY` / `CLUSTER BY`)
`seeds/memory/seasonality_calendar.csv`	Bible Â§22.3 â€" seed **Black Friday 2020** (start 2020-11-27, end 2020-11-30, `expected_dir=up`), **December peak** ("Christmas dip"/holiday, ~2020-12-01â†'2020-12-25), **New Year / January trough** (2020-12-26â†'2021-01-31, `expected_dir=down`) with `expected_mag` for confound subtraction
`seeds/memory/launch_calendar.csv`	Bible Â§22.3 â€" known GA4-sample launches, e.g. `landing_page=/sale` (`affected_dims`, `affected_metric`)
`seeds/memory/glossary.csv`	Bible Â§22.3 + CLAUDE.md Â§4 canonical names â€" synonym â†' `canonical_name` map (e.g. "checkout drop-off" â†' `checkout_to_purchase_rate`)
`scripts/init_memory.py`	Bible Â§22 â€" applies the DDL, loads the three seed CSVs, computes glossary/seasonality embeddings, initializes the vector store collection
`helios/mcp/report.py` (memory tools)	MCP_ARCHITECTURE Â§6.5 + Â§9. Add `save_diagnosis(diagnosis)` and `recall_prior(metric, segment)` to the existing `report-mcp` from M6b
`mcp_servers.yaml` (update)	MCP_ARCHITECTURE Â§3 â€" set `report-mcp`'s `helios_memory` dataset + `vector_store: ${HELIOS_VECTOR_STORE_URI}`

`models/semantic/semantic_layer.yaml`, `eval/scenarios/*.yaml`, `CLAUDE.md` **already exist â€" do not regenerate**.

Commands to run

```
source .venv/bin/activate
export HELIOS_VECTOR_STORE_URI=...      # e.g. local FAISS path or managed ANN endpoint

# 1) create the 8 tables in helios_memory
bq query --use_legacy_sql=false --location=US < sql/helios_memory_ddl.sql

# 2) seed calendars + glossary and build the vector collection
python scripts/init_memory.py --seeds seeds/memory --location US

# 3) start report-mcp with memory tools live (stdio)
python -m helios.mcp.report
```

```
# 4) round-trip smoke test
python -m helios.mcp.tests.memory_roundtrip   # save then recall the same finding
```

Tests to run

```
pytest helios/mcp/tests/test_report_memory.py -q
```

- Asserts `save_diagnosis(finding)` inserts one `diagnosis_history` row **and** upserts `(finding_id, embedding, metric, root_cause_label)` into the vector store; re-saving the same `finding_id` **upserts, not duplicates** (idempotent on `finding_id`, MCP_ARCHITECTURE Â§8).
- Asserts `recall_prior(metric, segment)` returns the just-saved prior via the hybrid query (exact filter on `metric` + `dimension_slice` â^a vector ANN), with recency weight `exp(-age_days/60)` applied.
- Asserts `seasonality_calendar` contains the BF2020 / December / January rows after seeding (a `SELECT COUNT(*)` â‰¥ 3 with the expected `event_label`s).
- Asserts `save_diagnosis` with a malformed envelope raises `ValidationError`.

Success criteria

- All 8 tables exist in `helios_memory` (`bq ls helios_memory` shows them with the Â§22 partition/cluster specs).
- `save_diagnosis â†' recall_prior` round-trips: a saved finding is retrievable by `(metric, segment)` and by vector similarity.
- `seasonality_calendar` is seeded with Black Friday 2020, the December peak, and the January trough (the priors the Critic needs at M9).
- `report-mcp` is the **only** memory writer â€" no agent and no other server holds a `helios_memory` write path.

Common mistakes

- **Not seeding the seasonality calendar.** Without BF2020/Dec/Jan rows, the M9 Critic is blind to known seasonality and will PASS expected moves as findings (Bible Â§22.3). Seed these first.
- **Suppression rows with no TTL.** `suppression_list.expires_at` defaults to a 30-day TTL so a re-emerging issue eventually re-surfaces; a permanent `NULL` should be deliberate, not the default â€" otherwise issues never come back.
- **Letting the LLM write memory directly.** All memory I/O goes through `report-mcp`; do not give any agent a raw `helios_memory` insert path (Bible Â§22, AGENT_ARCHITECTURE Â§8).
- **Giving `report-mcp` a BigQuery *query* client for analysis.** It reads/writes `helios_memory` only; it is **not** an analytics path and must never run governed marts SQL (least-privilege, MCP_ARCHITECTURE Â§9). Keep its scope to memory tables + vector store.
- **Non-idempotent `save_diagnosis`.** Key the upsert on `finding_id`; re-running a window must upsert, not double-write (MCP_ARCHITECTURE Â§8).
- **Hard-deleting stale priors.** Decay with `exp(-age_days/60)`; never delete (audit requirement, Bible Â§22.1). `expires_at` reads filter the suppression list; they don't drop history rows.
- **Wrong dataset location.** `helios_memory` must be created with `--location=US` to join the US-resident marts; a mismatched location yields "dataset not found".

M9 â€" Full 7-Agent Loop (L2 core)

Objective

Complete the deterministic FSM and the five remaining agents so a run produces **Critic-approved** findings, each carrying a decomposition, a significance test, a dollar revenue-at-risk, and a prescribed powered experiment. Add **Decompose** (Sonnet), **Diagnose** (Opus, hypothesis-tree RCA), **Critic** (Opus, adversarial refutation loop), **Prescribe** (Sonnet), and **Orchestrator** (Opus, drives the FSM + budget + routing). Extend `runner.py` to the full `PLANâ†'MONITORâ†'DECOMPOSEâ†'DIAGNOSEâ†'[CRITIC loop]â†'PRESCRIBEâ†'NARRATEâ†'END` machine with the Â§5.1 transitions, wire `A9.3` audit on every transition, and upgrade the Narrator to call `save_diagnosis` (now that M8 memory exists). Exit: the run yields PASS findings with significance + dollar impact +

an experiment card, and the FSM routes correctly (clean-run short-circuit; bounded DOWNGRADE re-query; DROPâ†'suppression) (AGENT_ARCHITECTURE Â§5â€"Â§10).

Files to create

Path	Copy/spec from
`helios/agents/decompose.py`	AGENT_ARCHITECTURE Â§6.3 â€" `DECOMPOSE_SPEC`; per anomaly `build_query` the `[{seg,w,r}]` table at t0/t1, call `stats.decompose_change` â†' `{mix,rate,interaction,by_segment}`, label `dominant`. Tools: `semantic.build_query`, `warehouse.dry_run`, `warehouse.run_query`, `stats.decompose_change`
`helios/agents/diagnose.py`	AGENT_ARCHITECTURE Â§6.4 â€" `DIAGNOSE_SPEC` + the best-first hypothesis-tree (expand by rate-effect magnitude; prune on `MIN_RATE_EFFECT` / `SIGNIFICANCE_ALPHA` / `MIN_SEGMENT_SESSIONS`; bounds `MAX_BRANCHING` / `MAX_DEPTH`; leaf promotion requires significant + single-dim + `reconcile` â‰¤0.5% + dollar-material; QUANTIFY `revenue_at_risk=(conv_t0â^'conv_t1)Ã—sessions_t1Ã—aov`)
`helios/agents/critic.py`	AGENT_ARCHITECTURE Â§6.5 â€" `CRITIC_SPEC` + the 4-axis refutation battery (mix-shift confound, insufficient sample, seasonality via `recall_prior` + `seasonality_calendar`, data-quality probe) â†' verdict PASS/DOWNGRADE/DROP. Tools: `semantic.build_query`, `warehouse.dry_run/run_query/reconcile`, `stats.significance_test`, `stats.decompose_change`, `report.recall_prior`
`helios/agents/prescribe.py`	AGENT_ARCHITECTURE Â§6.6 â€" `PRESCRIBE_SPEC`; `power_analysisâ†'runtime_estimateâ†'design_experiment`, rank by ICE, write `action_tracking(proposed)`. Tools: `experiment.power_analysis`, `experiment.runtime_estimate`, `experiment.design_experiment`, `semantic.get_metric`
`helios/agents/orchestrator.py`	AGENT_ARCHITECTURE Â§6.1 â€" `ORCHESTRATOR_SPEC`; open `run_state(PLAN)`, `recall_prior` priors+suppression, set scope/budget, drive FSM, route every candidate to Critic, finalize/ABORT. Tools: `warehouse.list_tables/describe_table`, `semantic.list_dimensions`, `report.recall_prior`
`helios/runner.py` (extend)	AGENT_ARCHITECTURE Â§5 (full FSM + Â§5.1 table) + Â§10 trace â€" add DECOMPOSE/DIAGNOSE/CRITIC-loop/PRESCRIBE states; Critic loop bounded by `MAX_REFUTE_ROUNDS`
`helios/agents/narrator.py` (upgrade)	AGENT_ARCHITECTURE Â§6.7 â€" add `report.save_diagnosis` + `report.export` to the M7 Narrator (faithfulness rule: every number traces to a tool-output hash)
`helios/agents/audit.py` (`A9.3`)	AGENT_ARCHITECTURE Â§4.2 + Bible Â§22.5 â€" `append_audit_log(run_id, step_seq, agent, mcp_tool, args_hash, sql_text, bytes_scanned, latency_ms, verdict, ts)` wired into the framework wrapper

`semantic_layer.yaml`, `eval/scenarios/*.yaml`, `CLAUDE.md` **already exist â€" do not regenerate**.

Commands to run

```
source .venv/bin/activate

# 1) node + FSM + Critic unit tests with recorded fixtures
pytest helios/agents/tests -q

# 2) a full real run (Orchestrator-driven) over a window with a known dip
python -m helios.runner --full --t0 2020-12-01 --t1 2020-12-20 \
  --window-t1b 2021-01-10 --metrics session_conversion_rate \
  --budget-gib 5 --time-budget-min 5

# 3) verify routing + persistence
python -m helios.runner --audit-check --run-id <last_run_id>   # 0 hand-authored SQL
```

```
bq query --use_legacy_sql=false --location=US \
  'SELECT state, n_findings, n_passed, n_dropped FROM helios_memory.run_state ORDER BY started_at DESC LIMIT 1'
```

Tests to run

```
pytest helios/agents/tests/test_diagnose.py -q     # tree expands by rate-effect, prunes correctly
pytest helios/agents/tests/test_critic.py -q       # PASS/DOWNGRADE/DROP on labeled fixtures
pytest helios/agents/tests/test_fsm_full.py -q     # all Â§5.1 transitions
```

- `test_critic.py` : a **pure mix-shift** finding mislabeled as a rate change â†' `DROP` or `DOWNGRADE` ; a finding overlapping a seeded `seasonality_calendar` entry within `expected_mag` â†' `DROP` / `DOWNGRADE` as expected; a clean significant single-dimension finding â†' `PASS` (AGENT_ARCHITECTURE Â§6.5, Â§12).
- `test_fsm_full.py` : clean run (0 anomalies) short-circuits MONITORâ†'NARRATEâ†'END; `DOWNGRADE` routes back to DIAGNOSE and is bounded by `MAX_REFUTE_ROUNDS=2` (after the cap â†' watchlist note, not a confident finding); `DROP` adds a run-local suppression so siblings don't re-raise; a hard failure/budget-exhaustion â†' `ABORTED` with an audit row and **no partial brief**.
- `test_diagnose.py` : drills the largest **rate-effect** slice first (not mix); prunes branches below `MIN_RATE_EFFECT` , above `SIGNIFICANCE_ALPHA` , or under `MIN_SEGMENT_SESSIONS` ; promotes a leaf only if reconciled (â‰¤0.5% drift) and dollar-material.

Success criteria

- A real run yields â‰¥1 **PASS** `Finding` , each carrying `decomposition` (mix/rate/interaction), `significance` (test + p + CI), `dollar_impact.revenue_at_risk_usd` , and a `recommendation.experiment_card_id` â€" the "every finding is actionable" rule (CLAUDE.md Â§3).
- FSM routing verified: clean-run short-circuits; `DOWNGRADE` re-queries are bounded by `MAX_REFUTE_ROUNDS` ; `DROP` writes run-local suppression; failure â†' `ABORTED` with no partial brief.
- The audit log proves **0 hand-authored SQL** and every brief number maps to a tool-output hash (faithfulness, Â§12).
- `save_diagnosis` persists each PASS (and DROP, for audit) to `diagnosis_history` ; Prescribe writes `action_tracking(proposed)` .
- `run_state` row closes at `END` with populated `n_findings/n_passed/n_dropped` and `brief_uri` .

(Quantitative accuracy â€" root-cause top-1 â‰¥85% vs â‰¤45% baseline, decomposition MAPE â‰¤10%, hallucination = 0 â€" is graded by the eval harness in **M10**, not asserted here.)

Common mistakes

- **Skipping the Critic / shipping unverified findings.** Every candidate must survive the Â§6.5 refutation battery before it reaches Prescribe/Narrator. A finding that bypasses the Critic is a defect.
- **The Critic confirming instead of refuting.** Its prompt and battery must *attack* the finding (default to skepticism); PASS only when all four refutations fail. A "looks plausible" Critic is useless.
- **Unbounded DOWNGRADE loops.** Cap Criticâ†'Diagnose re-query at `MAX_REFUTE_ROUNDS=2` ; after the cap, demote to a watchlist note, do not loop forever (Â§9, Â§5.1).
- **Drilling mix before rate.** Diagnose must order expansion by *rate-effect* magnitude â€" chasing mix effects first re-introduces Simpson's-paradox artifacts the decomposition exists to dissolve (Â§6.4, CLAUDE.md Â§4).
- **Promoting a leaf without `reconcile` .** Leaf promotion requires a `reconcile` â‰¤0.5% drift (G4) and a non-trivial dollar impact; skipping reconcile ships numbers that don't tie to canonical totals.
- **LLM authoring SQL or computing stats.** No agent holds a raw-SQL tool; all SQL is `semantic.build_query` , all math is `stats-mcp` / `experiment-mcp` (G1/G2). The framework allow-list makes raw SQL structurally impossible â€" keep it that way.
- **Dumping the hypothesis tree's row-level results into context.** Summarize intermediate nodes to `(slice, rate_effect, p, n)` tuples (Â§4.4); the tree widens fast and unsummarized rows blow the token/cost budget.
- **Not enforcing per-agent allow-lists in the full loop.** Each of the five new agents gets exactly its Â§2.1 row â€" e.g. the Narrator still cannot call `run_query` , Prescribe cannot touch `warehouse` . Enforce structurally, not by prompt.

M10 - Evaluation Harness & CI (L2 EXIT)

Objective

Prove the central empirical claim - **root-cause segment accuracy >= 85% vs <= 45% for the naive baseline**, with **0 hallucinated columns** - on an offline, *labeled* benchmark, and wire it into GitHub Actions as a hard regression firewall. The benchmark works by injecting synthetic-but-known anomalies into a frozen clone of the GA4 data (`helios_eval.fct_daily_funnel_perturbed`), running the full 7-agent FSM against the perturbed copy, and grading its diagnosis against ground truth recorded at injection time. This is the L2 exit: once green, no future change may drop top-1 accuracy more than `regression_tolerance` or introduce any hallucination. Grade STATISTICALLY (aggregate scores), never on byte-identical LLM output. See **Bible section 20** (Evaluation Framework) end to end.

Files to create

Path	Copy from
`helios/eval/injector.py`	Bible section 20.2 (injection mechanism: rate vs volume/mix primitives, seeded, writes `labels`)
`helios/eval/runner.py`	Bible section 20.6 (the `run_benchmark` core loop)
`helios/eval/report.py`	Bible section 20.9 (results-table template) + per-scenario JSON
`helios/eval/scorers/rootcause.py`	Bible section 20.4 / 20.7 (top-1 + top-3 on normalized segment key)
`helios/eval/scorers/decomposition.py`	Bible section 20.4 (MAPE on mix/rate/interaction, controls excluded)
`helios/eval/scorers/detection.py`	Bible section 20.4 (precision/recall/F1 over `(scenario, day, metric)` cells)
`helios/eval/scorers/dollars.py`	Bible section 20.4 (abs % error vs label `dollar_at_risk_usd`)
`helios/eval/scorers/hallucination.py`	Bible section 20.4 (SQL AST vs semantic-mcp registry + GA4 schema; hard zero)
`helios/eval/scorers/faithfulness.py`	Bible section 20.7 (two checks: numeric-claim hash rule + Critic-as-judge entailment)
`eval/gates.yaml`	Bible section 20.8 (thresholds, below)
`eval/baselines/main.json`	committed `main` -branch scores (seed from your first green full run)
`.github/workflows/ci.yml`	Bible section 20.8 (dbt build + tests + smoke-on-push / full-on-PR gate)
`eval/scenarios/*.yaml`	**ALREADY EXISTS** - 50 labeled scenarios across 7 buckets (`scenarios.yaml` + the 7 bucket files). **Do not regenerate.**

`eval/gates.yaml` (verbatim from Bible section 20.8):

```yaml
gates:
  rootcause_top1_min: 0.85
  hallucination_rate_max: 0.00     # hard zero
  decomposition_mape_max: 0.10
  detection_f1_min: 0.85
  dollar_error_max: 0.15
  faithfulness_min: 0.95
  regression_tolerance: 0.02       # top-1 may not drop >2pts vs main baseline
```

The shipped benchmark is **50 scenarios across 7 buckets** (the live `eval/scenarios/` directory): single_segment_rate (10), single_segment_mix (10), multi_segment_rate (6), multi_segment_mixed (6), seasonality_decoy (6), no_anomaly_control (6), data_quality (6). The decoy and control buckets are what separate Helios from the baseline - they punish over-flagging.

Commands to run

```
# POSIX: source .venv/bin/activate   |   Windows: .venv\Scripts\activate

# 0) materialize the frozen baseline clone the injector perturbs (one-time)
python -m helios.eval.injector --build-base \
```

```
  --src helios.marts.fct_daily_funnel \
  --dst helios_eval.fct_daily_funnel_base

# 1) smoke: 12-scenario subset, runs on every push (cost-bounded)
python -m helios.eval.runner --smoke

# 2) full: all 50 scenarios, runs on PRs to main
python -m helios.eval.runner --full

# 3) render the results table + per-scenario JSON artifacts
python -m helios.eval.report --run-dir eval/history/$(date +%Y%m%d-%H%M)

# 4) seed the committed baseline from your first green full run
cp eval/history/<run>/summary.json eval/baselines/main.json
```

CI invokes the same entrypoints - smoke on push, full on PR to `main` - then compares against `eval/baselines/main.json`.

Tests to run

```
# scorer unit tests (deterministic; recorded fixtures, no live BigQuery)
pytest helios/eval/scorers/tests -v

# golden decomposition check reused here: mix=-0.0018, rate=0, interaction=0
pytest helios/eval/scorers/tests/test_decomposition.py::test_mape_zero_on_golden -v

# end-to-end smoke (drives the FSM against perturbed data)
python -m helios.eval.runner --smoke
```

Asserts: scorers return correct sub-scores on fixtures; `decomposition.py` yields MAPE 0 when the predicted split equals the analytic golden; `hallucination.py` returns a non-zero rate for any AST node not in the registry; the smoke run produces a `summary.json` with all seven metrics populated and every gate evaluated.

Success criteria

- Full run prints **root-cause top-1 >= 0.85** while the naive baseline ("largest absolute segment delta", Bible section 20.5) scores **~0.45** on the same scenarios - the headline near-doubling.
- **Hallucination rate == 0.000** (hard gate; any non-zero fails CI regardless of accuracy).
- Decomposition MAPE <= 0.10; detection F1 >= 0.85; dollar-at-risk error <= 0.15; faithfulness >= 0.95.
- Total scanned bytes per full run **under 5 GiB** (the harness dry_runs every scenario first and aborts if the per-run budget is exceeded).
- CI gate is green: PR fails if any threshold is breached OR top-1 regresses more than `regression_tolerance` (0.02) vs `eval/baselines/main.json`.
- Each scenario emits a per-scenario JSON (predicted vs label, every sub-score) under `eval/history/` for debugging.

Common mistakes

- **Grading on live (unlabeled) data** instead of injected synthetic labels - the public sample's real anomalies have no known root cause, so you cannot compute accuracy. Always grade against `helios_eval.labels`, never the public source.
- **No naive baseline to beat** - without the 45% baseline computed on the *same* scenarios, "85%" is meaningless. The baseline must run in the harness, not be a remembered constant.
- **Eval not wired into CI** -> silent regressions. The benchmark must be a *required* check on any PR touching `models/`, `semantic/`, `agents/`, or `eval/`.
- **Hallucination gate not hard-zero** - setting `hallucination_rate_max` above 0 defeats grounding-over-generation; keep it `0.00` and AST-check emitted SQL against the registry + GA4 schema.
- **Filename drift** - `mcp_servers.yaml` and `hallucination.py` must point at `models/semantic/semantic_layer.yaml` (not `semantic_models.yml`); a stale path makes the AST check pass everything.
- **Cost blowup** - forgetting to dry_run every scenario, or letting the harness `SELECT *` over the perturbed shards. Cap bytes and abort over budget; the smoke subset (12) keeps push-time CI cheap.
- **Expecting byte-identical LLM output** - do not diff prose. Seed stats (`rng_seed 1729`), freeze the dbt model SHA + registry hash, and grade the *aggregate* metrics with tolerance.

- **Building M10 before M9 is green** - the eval harness *is* the agent layer's real grade; if the FSM isn't producing PASS findings with significance + dollars, every scorer reads garbage.

M11 - Autonomy & Depth (L3 EXIT)

Objective

Make Helios genuinely autonomous and statistically deeper: run on a schedule with no human in the loop, add forecast-residual anomaly detection plus cohort/RFM behavioral segmentation feeding the Diagnose hypothesis tree, expand the Critic to its full four-axis refutation battery, and add a secondary drill-down interface. The product's heartbeat is the **scheduled autonomous run** (conversation is a secondary surface). Exit when time-to-diagnosis holds **< 5 min/run on schedule**, accuracy holds **>= 85% across all canonical dimensions**, and memory-driven learning is demonstrable (a later run recognizes "seen before / already fixed / known-seasonal"). See **Bible section 23.3** (Phase 2 - Top-1% Undergrad / L3).

Files to create

Path	Copy from
`helios/scheduler.py`	Bible section 23.3 (scheduler entrypoint: a single bounded run, idempotent, writes `run_state`)
`deploy/scheduler.yaml` or `deploy/crontab`	Cloud Scheduler job (HTTP target) or cron line invoking `python -m helios.scheduler`
`helios/mcp/stats.py` (extend)	MCP_ARCHITECTURE section 6.3 - add `forecast` (prophet/pmdarima), `cohort_retention`, `rfm_segment`
`helios/agents/monitor.py` (extend)	AGENT_ARCHITECTURE section 6.1 - Monitor uses `forecast` residuals (expected-vs-actual) for detection
`helios/agents/diagnose.py` (extend)	AGENT_ARCHITECTURE section 6.4 - feed `cohort_retention` / `rfm_segment` slices into the hypothesis tree
`helios/agents/critic.py` (extend)	AGENT_ARCHITECTURE section 6.5 - full battery: mix-shift confound, insufficient sample, seasonality, data quality
`helios/drilldown.py`	Bible section 23.3 - secondary conversational drill-down over governed metrics (read-only, semantic-mcp only)
`macros/channel_group.sql` (harden)	DBT_GUIDE section 2 - session-scoped `event_params` source/medium first, `traffic_source` first-touch fallback

The full Critic battery is already specified in AGENT_ARCHITECTURE section 6.5 - copy its four refutations exactly: (1) re-run `decompose_change` and DROP if `dominant = mix`; (2) re-`significance_test` on an adjacent slice and DOWNGRADE if `p > alpha` or `n` small; (3) `recall_prior` + `seasonality_calendar` and DROP/DOWNGRADE if within `expected_mag` of a calendar entry overlapping the window; (4) data-quality confound probe (NULL spikes, `transaction_id` duplication, late shard) -> DROP if explained. Memory tools (`recall_prior`, `seasonality_calendar`) come from M8.

Commands to run

```
# 1) wire the new stats tools and re-validate the registry compiles clean
python scripts/validate_semantic.py            # 0 dangling references
python -m helios.mcp.stats                       # forecast / cohort_retention / rfm_segment expose

# 2) run one scheduled invocation by hand (must finish <5 min, write run_state)
python -m helios.scheduler --window last_28d

# 3a) Cloud Scheduler (GCP): daily 06:00 UTC, OIDC-authed HTTP target
gcloud scheduler jobs create http helios-daily \
  --schedule="0 6 * * *" --time-zone="UTC" \
```

```
  --uri="$HELIOS_RUN_URL" --http-method=POST \
  --oidc-service-account-email="helios-wh@$PROJECT.iam.gserviceaccount.com"

# 3b) or local cron (POSIX): same daily cadence
#   crontab -e  ->  0 6 * * *  cd /opt/helios && .venv/bin/python -m helios.scheduler

# 4) drill-down interface (secondary surface, read-only)
python -m helios.drilldown --ask "checkout_to_purchase_rate by device_category last_14d"
```

Tests to run

```
# forecast residual + behavioral segmentation tool tests
pytest helios/mcp/tests/test_stats.py -k "forecast or cohort or rfm" -v

# full Critic battery: each axis DROPs/DOWNGRADEs the right confound
pytest helios/agents/tests/test_critic.py -v
#   asserts: pure mix labeled as rate -> DROP; small-n -> DOWNGRADE;
#   known-seasonal move within expected_mag -> DROP; NULL-spike confound -> DROP;
#   a clean finding -> PASS

# accuracy holds across all canonical dimensions (full eval re-run)
python -m helios.eval.runner --full

# scheduled-run latency + idempotency
pytest helios/tests/test_scheduler.py -k "under_5min and idempotent" -v
```

Success criteria

- A scheduled invocation completes a full diagnosis in **< 5 min/run**, sustained across repeated scheduled fires, and writes `run_state` so re-fires are idempotent.
- Eval top-1 holds **>= 85%** when broken out **per canonical dimension** (device_category, channel_group, country, landing_page) - not just in aggregate.
- The Critic catches each adversarial axis: the seasonality_decoy bucket and data_quality bucket are NOT flagged as behavior changes (refuted), keeping detection precision high on the control/decoy scenarios.
- Forecast-residual Monitor reduces false positives on the no_anomaly_control bucket vs the M9 threshold detector (measurable drop in false-flag rate).
- **Memory-driven learning is demonstrable:** a second run over an overlapping window recalls a prior diagnosis (`recall_prior`) and either suppresses an already-fixed/known-seasonal issue or down-weights a root cause whose fix already won (`action_tracking.outcome = win`).

Common mistakes

- **Not seeding** `seasonality_calendar` (M8) -> the Critic is blind to known seasonality and flags the post-holiday dip in the decoy bucket as a real anomaly, tanking precision. Seed BF2020/Dec/Jan before relying on the seasonality refutation.
- `suppression_list` **with no TTL** -> issues never resurface; a transient drop gets permanently muted. Every suppression carries an expiry.
- **Using event-level** `traffic_source` **as the session channel source** - it is USER first-touch, not session-scoped. Harden `channel_group_case()` to read session-scoped `event_params` source/medium first, falling back to `traffic_source` only when null; do not duplicate channel logic outside the single macro.
- **Not seeding the RNG** (`rng_seed 1729`) in forecast/cohort/RFM -> non-reproducible runs and flaky eval. All stats math is deterministic, seeded code in stats-mcp.
- **Making the LLM the scheduler/controller** - the scheduled entrypoint is deterministic Python that bounds the run (window, byte budget, refute rounds); the LLM only fills agent steps.
- **Skipping the full Critic battery** to "save time" - shipping mix-labeled-as-rate findings is exactly the failure the 85%-vs-45% claim is supposed to disprove; an unrefuted finding is not a finding.
- **Letting drill-down author raw SQL** - the secondary interface must go through semantic-mcp like every other path (G1/G5); it is read-only and bounded, never an ad-hoc SQL chatbot (that is the anti-product).

M12 - Productionization & Frontier (deferred)

Objective

Capture the post-L3 roadmap honestly: what production-hardening and frontier capabilities look like, and why each is explicitly deferred on *this* dataset. Do **not** build M12 to pass the L1-L3 gates - it is out of scope for the standalone build and listed so the reader knows the ceiling and the rationale. See **Bible section 23.4** (Phase 3 - Productionization) and **section 23.5** (Phase 4 - Frontier), with deferral rationale in section 23.7.

Files to create

None required for the standalone build. The items below are design targets, not deliverables for the playbook's MVP-to-L3 path. If pursued later, they extend existing artifacts (warehouse-mcp adapters, per-tenant `semantic_layer.yaml`, the stats-mcp causal module) rather than replacing them.

Phase 3 - Productionization (Bible section 23.4):

- **Multi-tenant isolation** - per-tenant semantic layer + per-tenant byte budgets so one tenant cannot scan another's data or blow a shared budget.
- **Warehouse-agnostic adapters** behind `warehouse-mcp` (Snowflake / Databricks / DuckDB) so the semantic layer remains the *only* SQL author regardless of dialect.
- **Streaming ingestion** of GA4 events (intraday) feeding `fct_daily_funnel` near-real-time, plus observability (per-agent latency, token cost, cache hit-rate).
- *Exit gate:* two warehouses pass **identical reconcile tests**; p95 run latency under SLA at N tenants.

Phase 4 - Frontier (Bible section 23.5):

- **Causal inference** - difference-in-differences, synthetic control, double-ML replacing correlational decomposition where the data supports it.
- **Auto-executed experiments** - push hypothesis cards to an experimentation platform, read back results, auto-update the backlog (closing the loop the Prescribe agent opens).
- **Multi-dataset** - join GA4 with CRM/cost data for true CAC/ROAS.
- *Exit gate:* a causal estimate **validated against a held-out randomized experiment**.

Commands to run

None. M12 has no commands in the standalone build path.

Tests to run

None for the standalone build. When pursued: the production exit gate is the *identical-reconcile-across-two-warehouses* test; the frontier exit gate is the *causal-estimate-vs-held-out-experiment* validation.

Success criteria

- The roadmap is documented and the deferrals are justified - not silently dropped. The honest constraints (Bible section 23.7) are:
  - **True causal inference** is deferred: the public GA4 sample has **no experiment assignment**, so L1-L3 ship *correlational* decomposition with explicit confound caveats from the Critic. Do not claim causality you cannot support.
  - **Real-time streaming** is deferred: a 3-month static sample gains nothing from intraday latency; batch is sufficient.
  - **True customer LTV** and **cross-device identity** are deferred: the ~3-month window is too short for lifetime curves (report 30/60-day proxy retention only), and `user_id` is almost always NULL so identity stitching is unsound.
  - **ML-based attribution** is deferred: it needs session-scoped channel + cost data the sample lacks; use GA4 default channel grouping until then.

Common mistakes

- **Over-claiming causality** - presenting correlational decomposition as a causal estimate. The Critic's job (and the brief's caveats) is to keep claims honest until Phase 4 data exists.
- **Building frontier features to hit L2/L3 gates** - the 85%-vs-45% claim is won by grounding + decomposition + the Critic + the eval harness, not by uplift modeling. Do not gold-plate.
- **Hardcoding BigQuery in agent/stats code** - if production-hardening is ever pursued, the *only* SQL author must remain the semantic layer; leaking dialect-specific SQL into agents breaks the warehouse-agnostic exit gate before it starts.
- **Promising LTV / cross-device on this data** - both are unsound on the ~3-month, mostly-NULL-`user_id` sample; state the proxy (30/60-day retention) and move on.

Appendix A â€” Global Troubleshooting

Symptom	Likely cause	Fix
`dbt debug` fails to connect	wrong project/dataset/location or no ADC	set `location: US` (the GA4 sample is US), run `gcloud auth application-default login`, check `profiles.yml` project.
"Dataset not found: ga4_obfuscated_sample_ecommerce"	location mismatch	the public dataset is in **US**; your target dataset + job location must be US.
Query scans 100s of GB / huge bill	no shard pruning / `SELECT *`	always filter `_TABLE_SUFFIX BETWEEN ...`; set `maximum_bytes_billed`; never `SELECT *` over `events_*`.
Step rate > 1 (e.g. `view_to_cart_rate` = 1.4)	occurrence-based flags, not monotonic	use `reached_*` **max-downstream** flags (M3); rerun the monotonicity test.
`dim_channels` test fails	an 11th channel group crept in	keep **exactly 10** groups; channel logic only in `channel_group_case()`.
Revenue / AOV ~2Ã— too high	`transaction_id` not deduped	dedup orders by `transaction_id` in `fct_orders` (M4).
`revenue` â‰ `gross_revenue` totals	session-attributed vs order-grain	they reconcile only at the **grand total**; don't compare per-slice.
`validate_semantic.py` fails (dangling ref)	a metric's numerator/denominator/expr names a metric not in the registry	fix the reference or add the metric; the registry is the single source of truth.
`semantic-mcp` can't find the registry	filename drift	`mcp_servers.yaml` must point at `models/semantic/semantic_layer.yaml`.
`run_query` raises `NotDryRunFirst`	called without a prior `dry_run`	always `dry_run` first (this is the guardrail working â€” don't bypass it).
`decompose_change` golden test off	wrong mix/rate/interaction algebra	mix= `Î£ Î”wá¿¢Â·rá¿¢(t0)`, rate= `Î£ wá¿¢(t0)Â·Î”rá¿¢`, interaction= `Î£ Î”wá¿¢Â·Î”rá¿¢`; seed `rng_seed=1729`.
Eval accuracy looks great but agents hallucinate	grading on live data, no hallucination gate	grade against the **injected labels**; make hallucination a **hard-zero** CI gate.
Critic flags everything seasonal as real	`seasonality_calendar` not seeded	seed Black Friday 2020 / Dec peak / Jan trough (M8).
Run never finishes / huge token cost	row-level dumps into LLM context	summarize query results to aggregates before they enter the model; enforce per-agent tool allow-lists.
LLM output not byte-reproducible	expecting determinism from the model	the agent layer is graded **statistically** by the eval harness, not asserted byte-identical; only stats/SQL are deterministic.

Appendix B â€” Definition of Done (per level)

- **L1 (after M7):** governed marts + semantic layer + the minimal loop; `reconcile('revenue','day')` matches a hand-written control query to the cent; **0 hallucinated columns**; one anomaly â†' a Decision Brief in **<5 min**.
- **L2 (after M10):** all 7 agents + 5 MCP servers + the Critic + the eval harness in CI; every finding carries significance + dollar impact + an experiment; **root-cause â‰¥85% vs â‰¤45% baseline**; cost under the byte budget.
- **L3 (after M11):** autonomous scheduled runs with memory-driven learning; forecasting / cohorts / RFM; full Critic refutation battery; accuracy holds across all canonical dimensions.

Appendix C â€” Minimum-viable Helios (if you have limited time)

You do not have to build all of M0â€"M12 to have something valuable:

- **A weekend (M0â€"M5):** the governed dbt marts + the validated semantic layer. This alone is a strong analytics-engineering portfolio piece â€" correct, tested, documented metrics on real GA4 data, queryable by anyone. **No LLM required.**
- **A week (+ M6, M6b, M7):** add the MCP servers and the minimal loop â†' governed SQL + deterministic stats + a single automated Decision Brief. This is the "grounded AI analyst" story end-to-end on one path.
- **Twoâ€"three weeks (+ M8â€"M10):** the full 7-agent loop and the benchmark â†' the headline **85%-vs-45%** result, which is the project's central, defensible claim. This is the L2 portfolio centerpiece.

Build depth-first along the critical path (`DEPENDENCY_MAP.md` Â§4), not breadth-first. Finance facts, cohorts, the scheduler, and the drill-down UI can all wait.

Appendix D â€" Build checklist

```
[ ] M0  repo + GCP/IAM + dbt config       (dbt debug green)
[ ] M1  sources + macros + seed           (dbt deps/seed)
[ ] M2  staging                           (staging tests green)
[ ] M3  sessionization + funnel KEYSTONE  (monotonicity + uniqueness golden tests)   â˜… test first
[ ] M4  marts                             (reconcile to the cent; channels = 10)
[ ] M5  semantic layer live               (validate_semantic.py â†' 0 dangling refs)
[ ] M6  semantic-mcp + warehouse-mcp      (round-trip + budget gate)
[ ] M6b stats + experiment + report MCP   (decompose_change golden test)
[ ] M7  minimal loop                      (anomaly â†' brief <5 min; 0 hallucination)   âœ… L1
[ ] M8  memory                            (save/recall; calendars seeded)
[ ] M9  full 7-agent loop                 (PASS findings: significance + $ + experiment)
[ ] M10 eval + CI                         (â‰¥85% vs â‰¤45%; hallucination 0)            âœ… L2
[ ] M11 autonomy + depth                  (<5 min/run scheduled; all dims)           âœ… L3
[ ] M12 productionization / frontier      (deferred)
```

Appendix E â€" Where the code lives (spec doc index)

`DBT_GUIDE.md` (all dbt code: config, sources, staging, intermediate, marts, tests, freshness, lineage, docs) Â· `DATA_MODEL.md` (tables, grains, keys, ER) Â· `models/semantic/semantic_layer.yaml` (the registry) Â· `MCP_ARCHITECTURE.md` (5 servers + skeletons) Â· `AGENT_ARCHITECTURE.md` (7 agents + FSM + Critic + RCA) Â· `eval/scenarios/scenarios.yaml` (50 labeled scenarios) Â· `METRIC_GOVERNANCE_GUIDE.md` + `METRIC_DEPENDENCY_GRAPH.md` (metric governance) Â· `DEPENDENCY_MAP.md` + `DEVELOPMENT_PLAN.md` (build order) Â· `CLAUDE.md` (conventions + grounding rules) Â· `HELIOS_PROJECT_BIBLE.md` (the full 25-section reference).

> If you build nothing else, build M0â€"M5 correctly. Everything downstream trusts those marts â€" and they fail *silently* if the keystones (M3) are wrong, so test them first and reconcile to the cent.